TASK 1: Oracle VirtualBox Setup and Ubuntu

Date: 12-02-2026

Oracle VM VirtualBox is a virtualization application that enables users to run different operating systems on one computer. This involved first installing Ubuntu Linux inside the

VirtualBox to create a virtual Linux environment. It is used for:

- Operating system and command line operation
- Create a secure and isolated environment for testing purposes

1. Installing Oracle VirtualBox

Steps:

1: Visit https://www.virtualbox.org
2: Download the installer for Windows hosts.
3: Open the installer file and click Next to start the setup process.
4: Keep the default configuration settings and install all the required components.
5: Allow the network driver installation.
6: Hit Finish to finalize the installation.

2. Download Ubuntu

Steps:

1: Go to https://ubuntu.com/download/desktop
2: Download the Ubuntu 22.04 LTS ISO file.

3. Creating Ubuntu Virtual Machine

Steps:

1: Open VirtualBox and click on New.
2: Enter the following information:

- **Name: Ubuntu**
- **Linux type**
- **Version: Ubuntu (64-bit)**

3: Allocate memory (RAM):

- **Minimum: 2048 MB**
- **Recommended: 4096 MB**

4: Set up a virtual hard disk:

- **Disk Type: VDI (VirtualBox Disk Image)**
- **Storage Allocation: Dynamically allocated**
- **Disk Size: Between 20–25 GB**

4. Installing Ubuntu

Steps:

1: Select the created virtual machine and click on Start.
2: Select the ISO with the download Ubuntu operating system.
3: Click Install Ubuntu.
4: Select the Normal installation option.
5: Select Erase disk and install Ubuntu (no changes will be made on your physical disk; alterations will be limited to the virtual disk only).
6: Provide a username and password.
7: When installation is done, the user should restart the virtual machine.

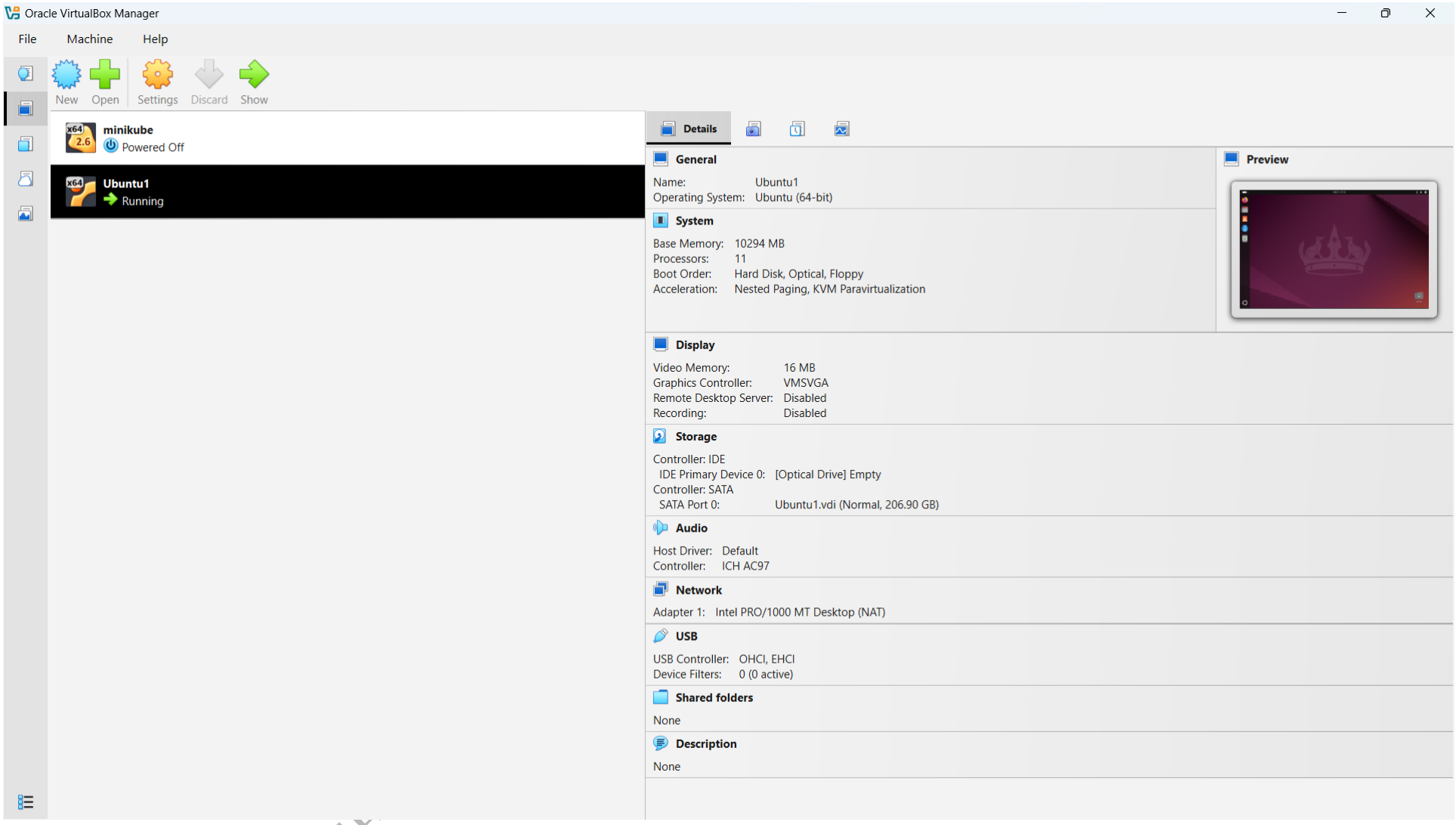

Figure 1: Ubuntu OS installed in a VM

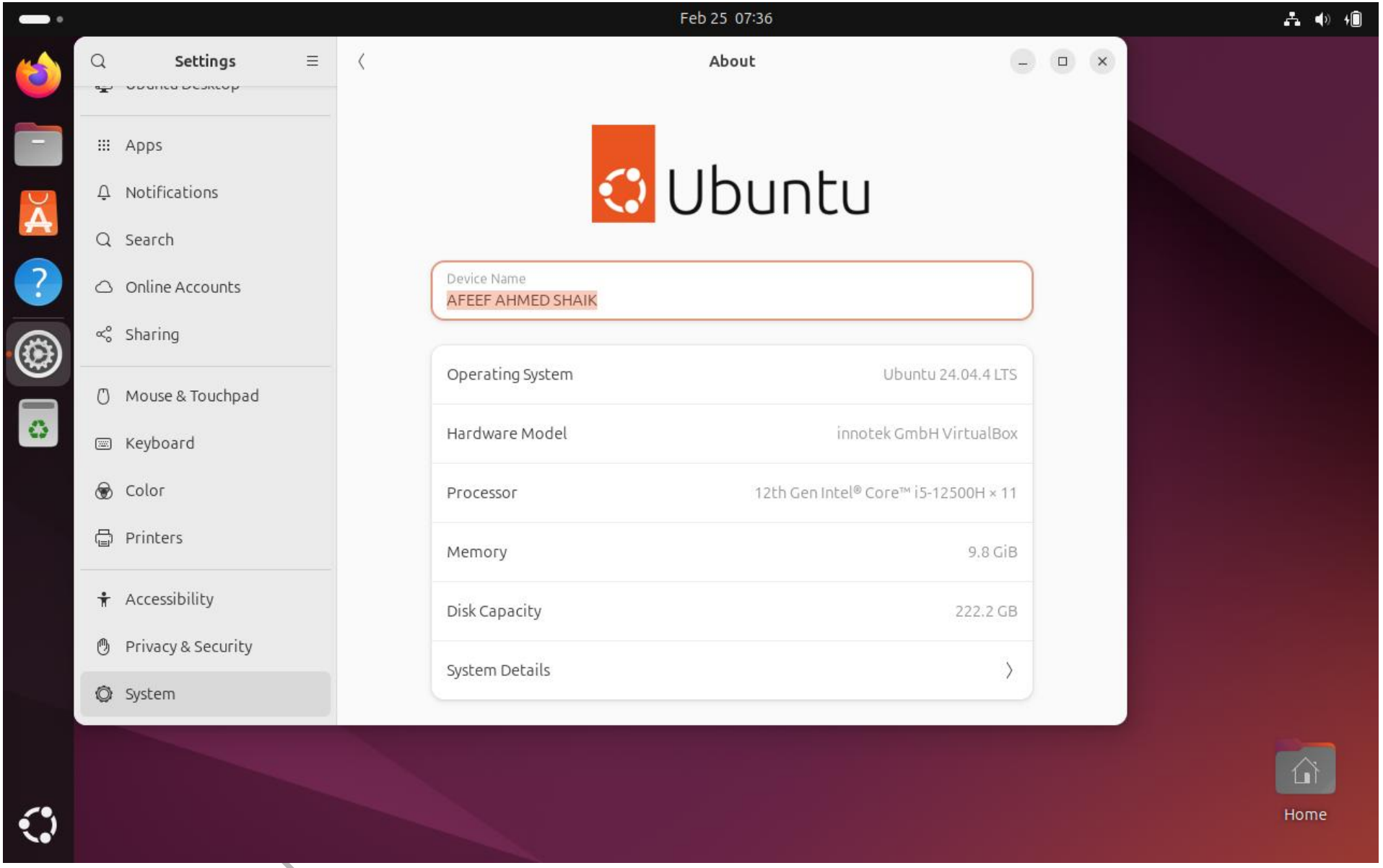

Figure 2: USER PROFILE

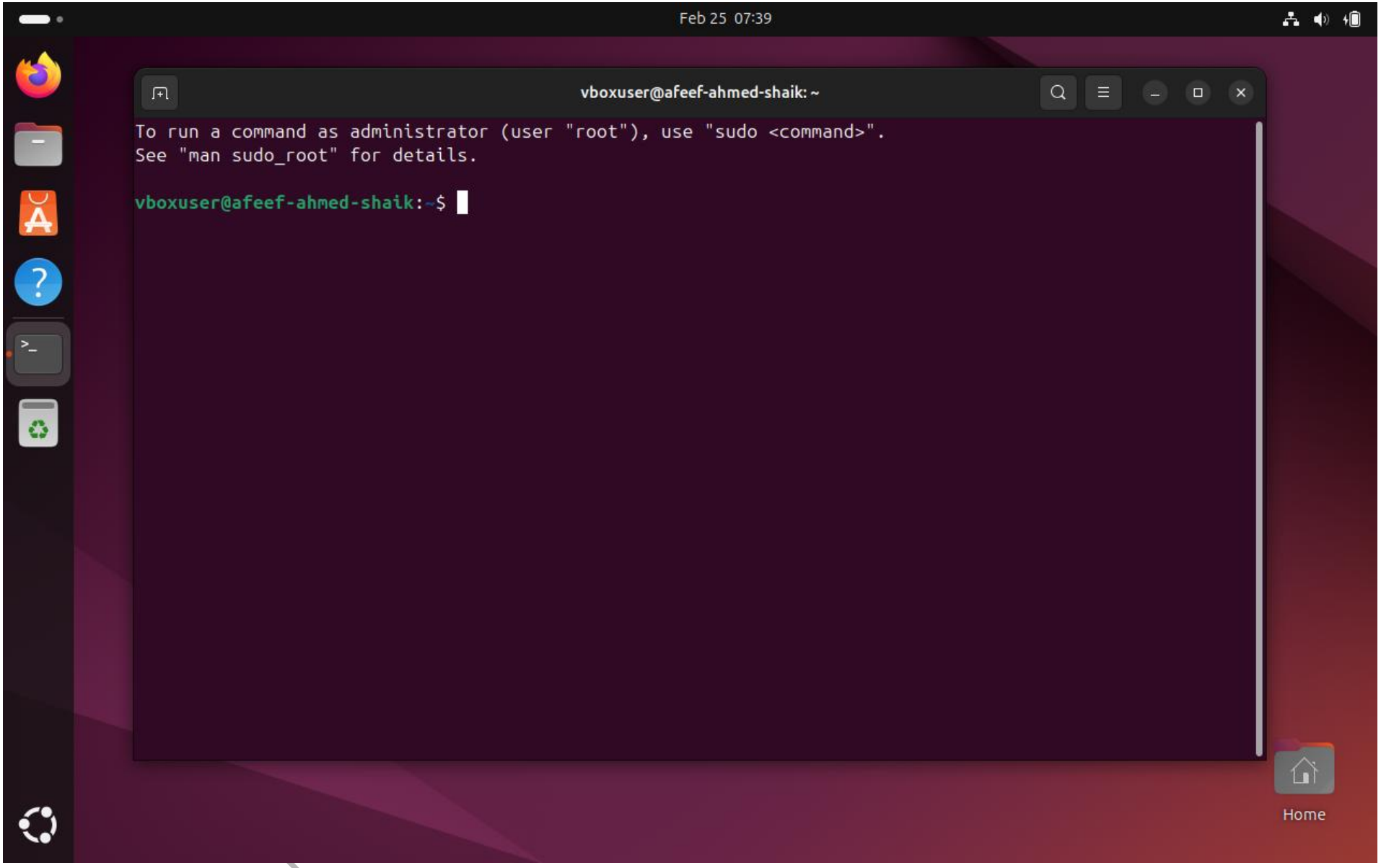

Figure 3: USER TERMINAL

Task 2 : Working Docker Images and NGC Container

Date: 18-02-2026

NVIDIA NGC Login and API Key Generation

To use GPU-enabled containers, which are available on the NVIDIA Container Registry, an **NVIDIA NGC** account was created. To ensure that the system can securely download the NGC containers, an API key has been generated and used to authenticate Docker.

Step 1: Create NVIDIA NGC Account

1. Launch a web browser.
2. Visit https://ngc.nvidia.com
3. Press **Sign In**.
4. Click on Create Account.
5. Fill out the registration form, and verify your email to activate your account.

Step 2: Generate NGC API Key

1. Log in to the NVIDIA NGC dashboard.
2. Click your User Profile icon.
3. Select **API Key** option.
4. Click Generate API Key.
5. Copy the generated key and save it somewhere secure for later use.

Step 3: Login to NGC Using Docker (Ubuntu Terminal)

In the Ubuntu terminal, run the following command: Code: docker
login nvcr.io Input the following credentials when asked

- **Username: $oauthtoken**
- **Password: Paste the generated NGC API key**

With a successful login process, the terminal will display: The terminal after a login sequence is as follows: Login Succeeded.

This shows that the system is successfully authenticated and can now pull containers from the NVIDIA NGC registry.

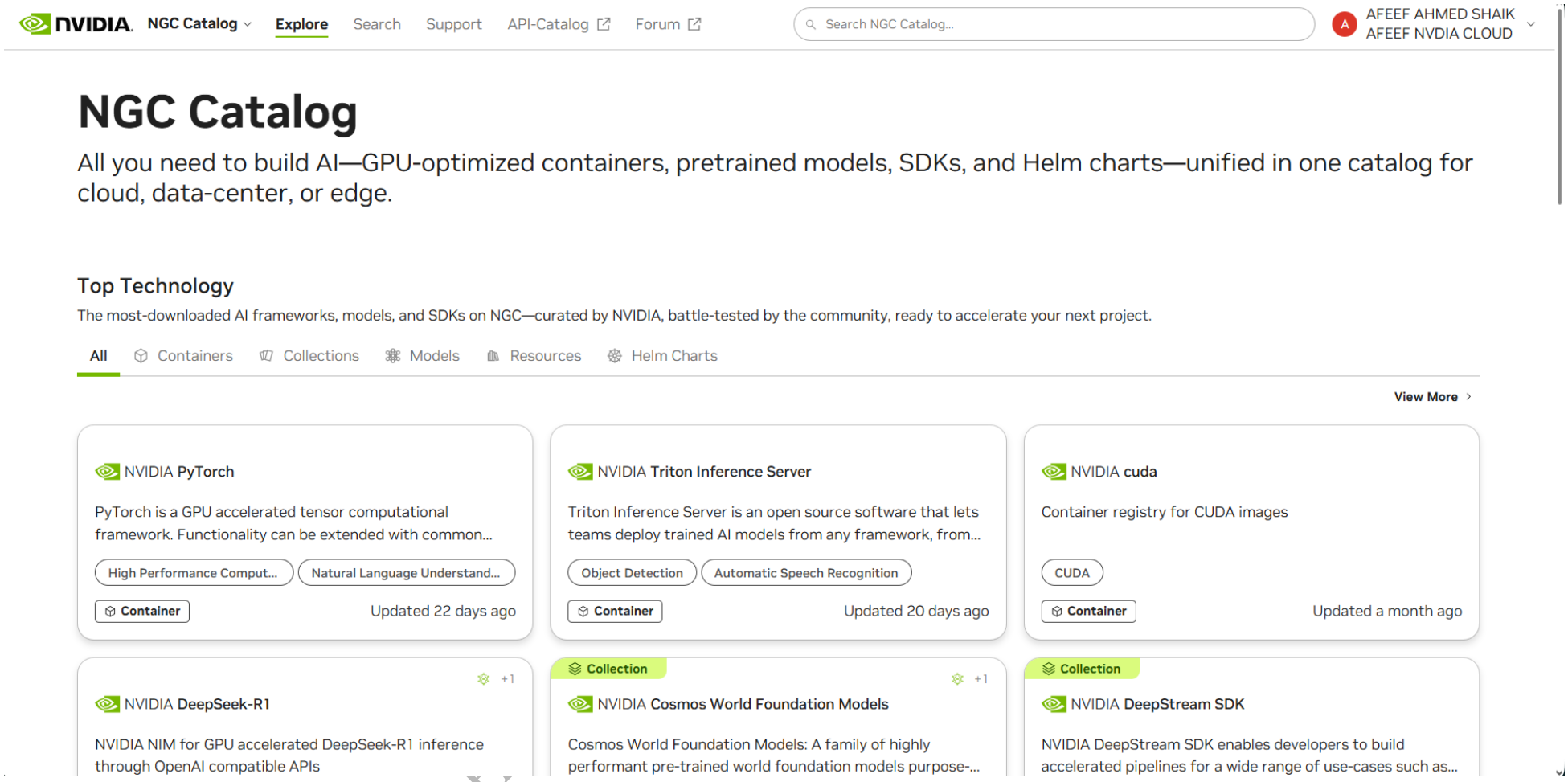

Figure 4: NVIDIA NGC USER DASHBOARD

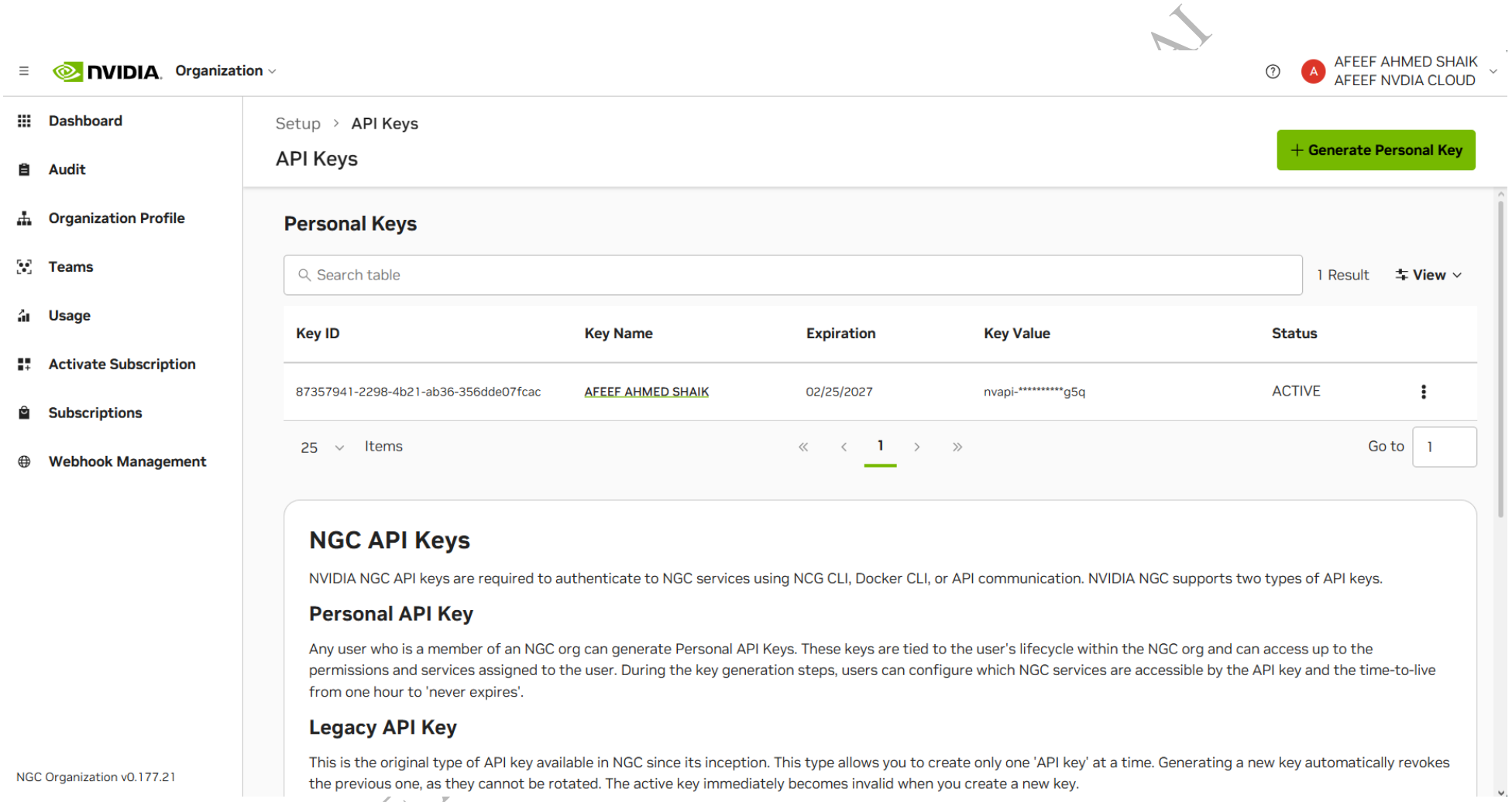

Figure 5: NVIDIA NGC API ORGANIZATION DASHBOARD

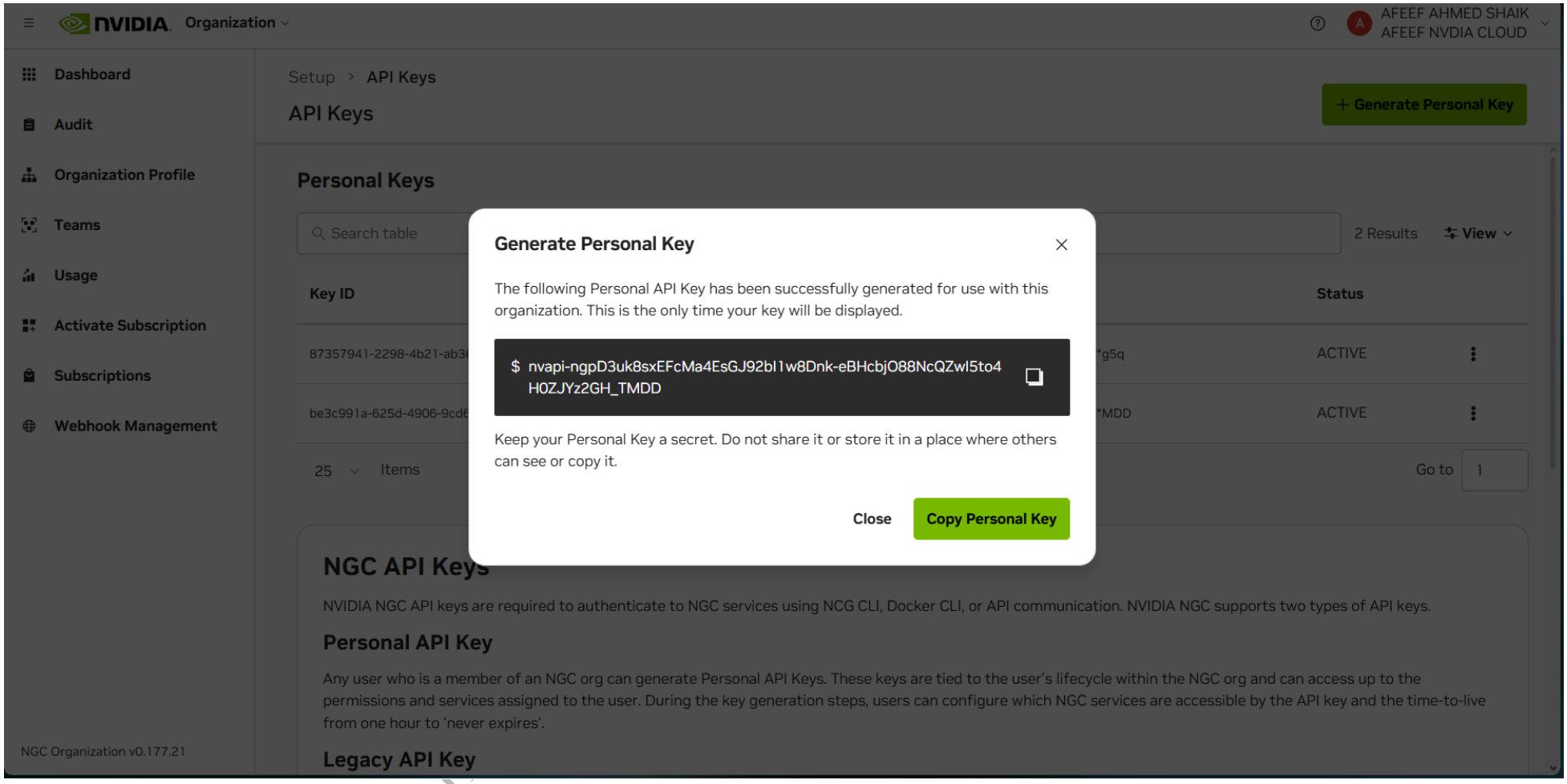

Figure 6: NVIDIA NGC API KEY GENERATION

Docker Images Before Pulling GitHub and Friend Images

The existing Docker images were checked first to ascertain the current status of the overall Docker trends before downloading any new images.

The below command was run:

```
docker images
```

This command lists out all the docker images stored locally in the system. These details include repository, tag, image ID, the date when the image was created, and the size of the image.

By running this command, verifying which images are already present and which have been downloaded later becomes much easier.

At this point, only previously pulled images (for example, NVIDIA container images) were in the system. Second, there were no custom images downloaded into the system created by the user or their peers.

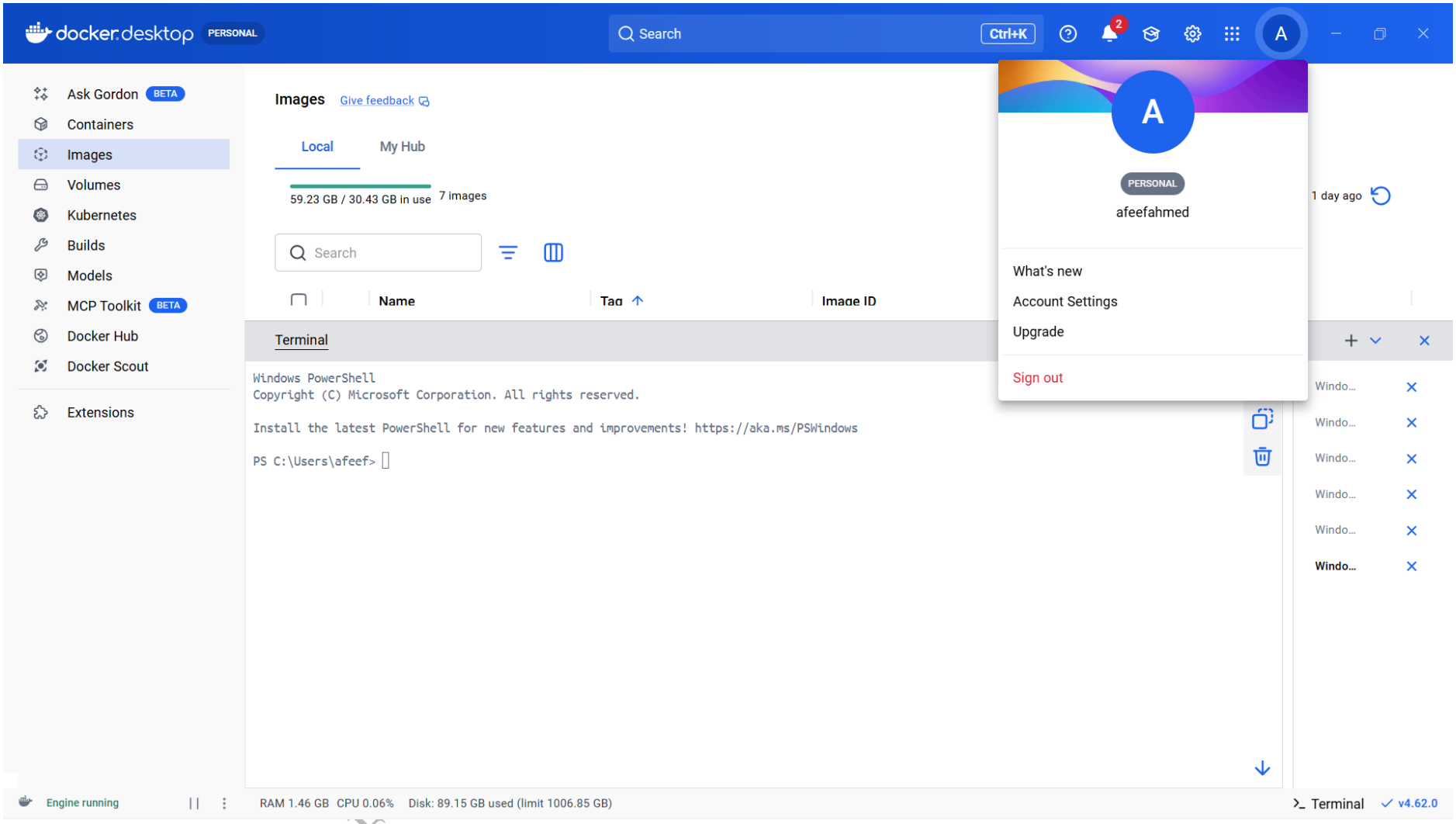

Figure 7: Docker Images Before Pulling GitHub and Friend Images

Pulling TensorFlow Container & Program Execution

The TensorFlow container was retrieved from the NVIDIA NGC registry to initiate a pre-configured deep learning Dockerized environment. This container has Python, TensorFlow, and all the necessary libraries needed in the process, hence, there is no need to install dependencies manually.

```
docker pull nvcr.io/nvidia/tensorflow:25.10-tf2-py3
```

After the downloading process had been completed, the container was then launched in interactive mode through:

```
docker run -it nvcr.io/nvidia/tensorflow:25.10-tf2-py3
```

This command started an interactive Python session inside the container.

To verify the TensorFlow installation, whether it was successful or not, and if the software behaved as required, the following commands were run in Python:

```
import tensorflow as tf
print("TensorFlow Version:", tf._version_)
print("GPU Available:", tf.config.list_physical_devices('GPU'))
```

The output included the currently installed tensorflow version and a statement that the container was running properly. Since the system does not have an NVIDIA GPU, the GPU list was empty, proving that the GPU has been disabled and that TensorFlow is working in CPU mode.

This verified that the TensorFlow container was successfully downloaded and run within the Docker environment.

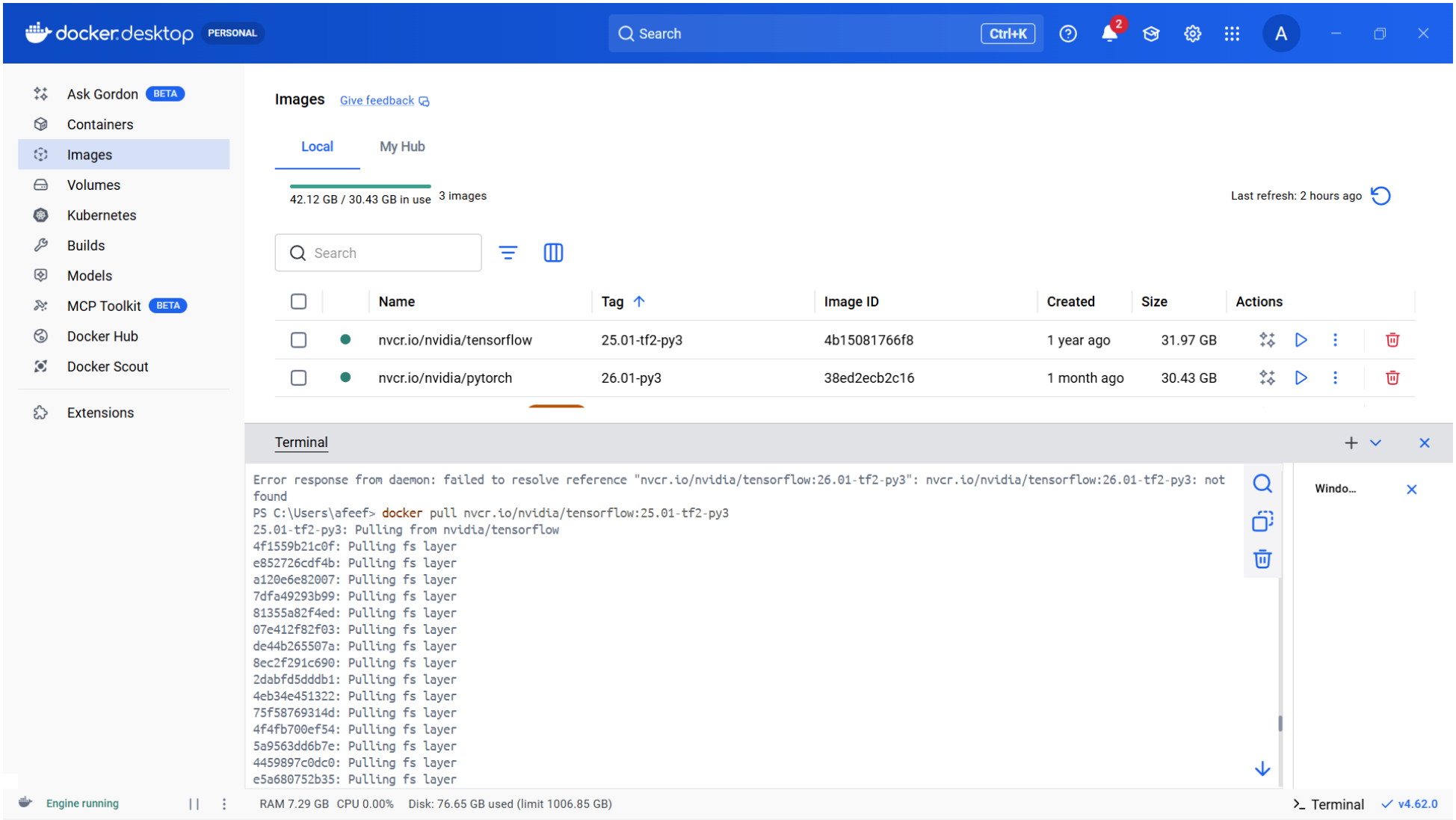

Figure 8: TensorFlow Container Pull

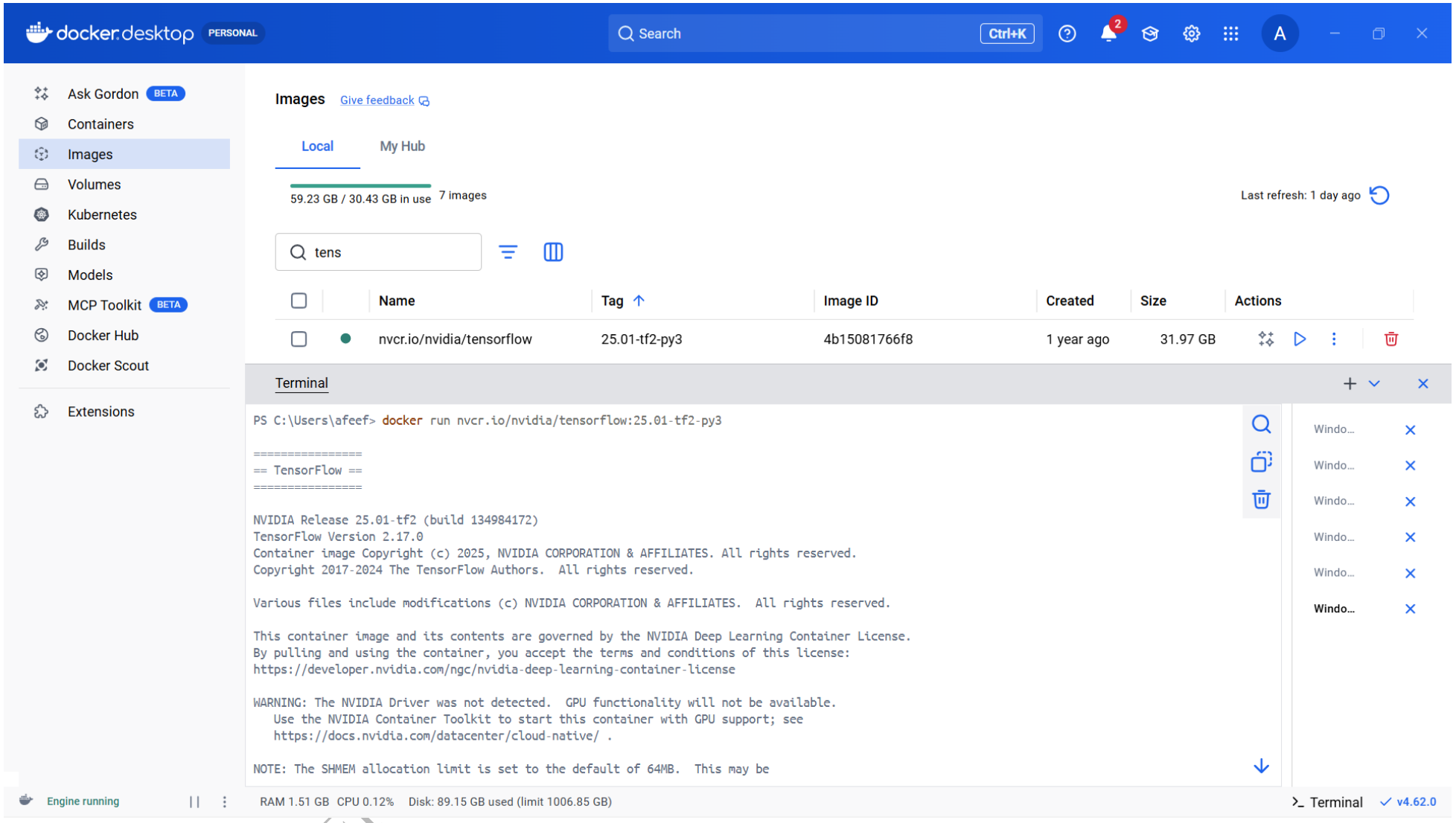

Figure 9: TensorFlow Container Run

PyTorch Container Pull & Program Execution

To have a ready deep learning environment inside Docker, the PyTorch container was downloaded from the NVIDIA NGC registry. Not only does this container require no manual configuration, but also it comes with Python, PyTorch, CUDA libraries (in case a compatible GPU is available), and all necessary dependencies.

The image was pulled using this command:

```
docker pull nvcr.io/nvidia/pytorch:26.10-py3
```

After the download was performed successfully, the container was launched in the interactive mode with:

```
docker run -it nvcr.io/nvidia/pytorch:26.10-py3
```

This command opened a Python terminal within the container environment.

To make sure that the PyTorch tool was correctly installed and working properly, the following Python commands were executed:

```
import torch
print("PyTorch Version:", torch._version_) print("GPU Available:",
torch.cuda.is_available())
```

The output showed the installed PyTorch version and the availability of GPU acceleration. Since the system does not an NVIDIA GPU, the result returned False, meaning that PyTorch was running on CPU.

This verified that the PyTorch container has been correctly downloaded and is running inside the Docker environment.

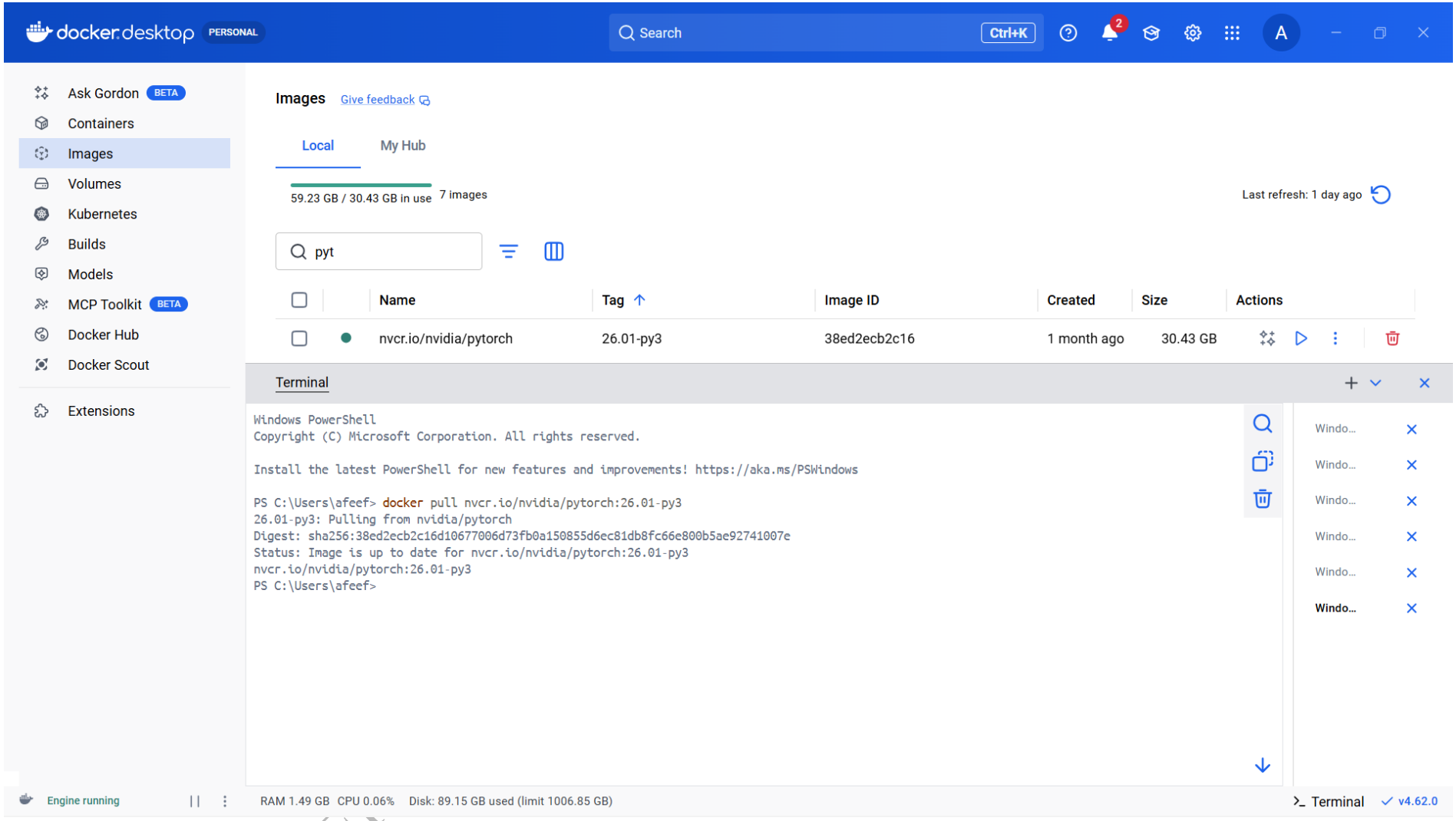

Figure 10: PyTorch Pull

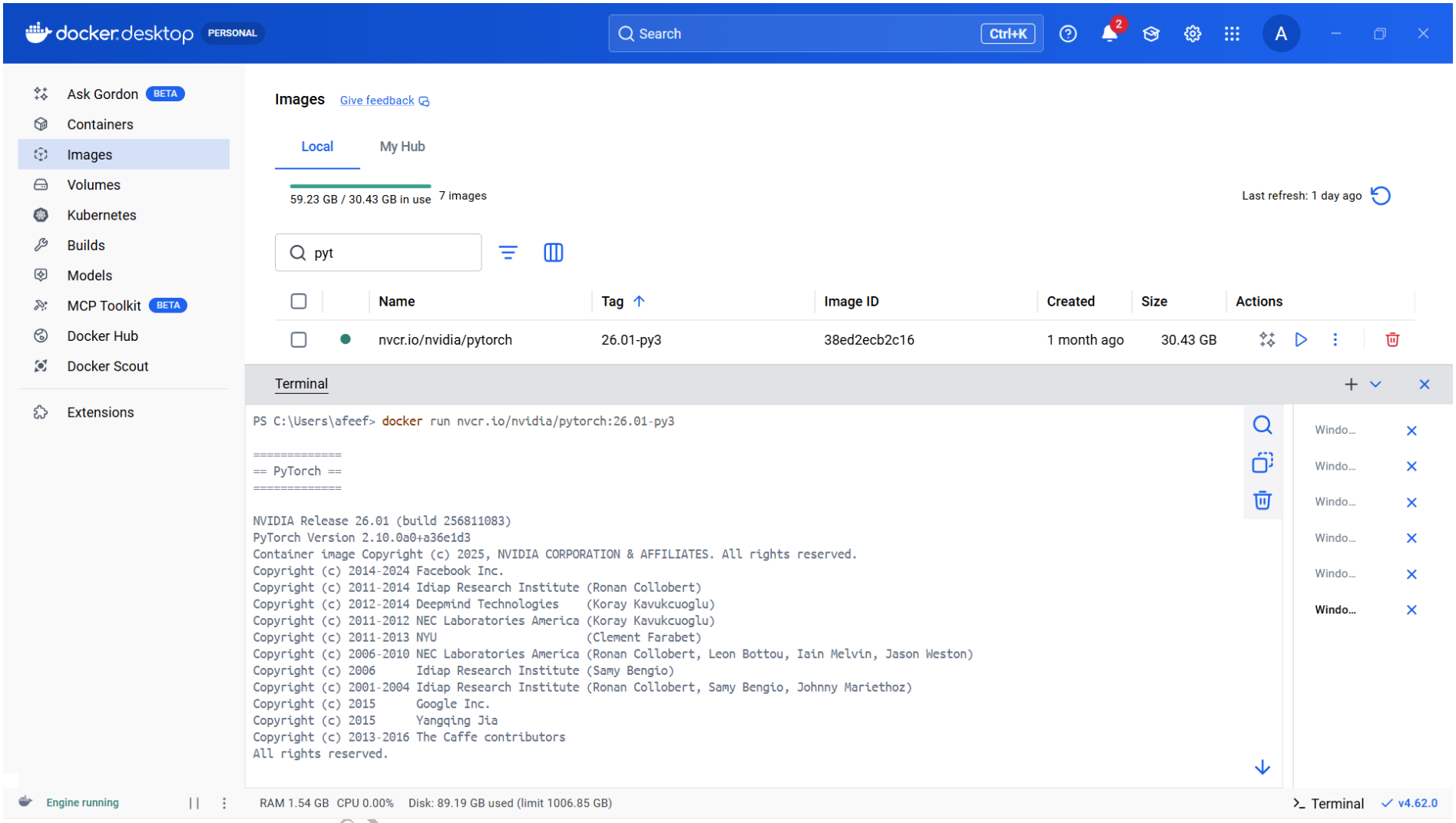

Figure 11: PyTorch Run

Use Case Container Setup Using NGC Pull and Run

As a practical example to show the downloading and execution processes of NGC containers with Docker, the Triton Inference Server container was used. It is built in with accelerated libraries for efficient, high-performance inference, and optimized to deploy and serve AI models in production settings.

The container image was pulled from the NVIDIA NGC registry using the below command; This is followed by pulling the specific container image from the NVIDIA NGC registry:

```
docker pull nvcr.io/nvidia/tritonserver:24.10-py3
```

After the image was downloaded successfully, the container was launched to confirm that it functions properly:

```
docker run nvcr.io/nvidia/tritonserver:24.10-py3
```

Once the container was started, initialization logs appeared on the terminal, implying that the Triton Inference Server was starting correctly. Thus, once the container started, initialization logs appeared on the terminal, and it was evident that the Triton Inference Server was starting successfully, and these logs verified that the container was functioning properly.

Triton Inference Server is common for deploying and managing trained machine learning models so that applications can send inference requests and get predictions in real time. The experiment has proven the capabilities of NGC container in real-world AI model serving and deployments.

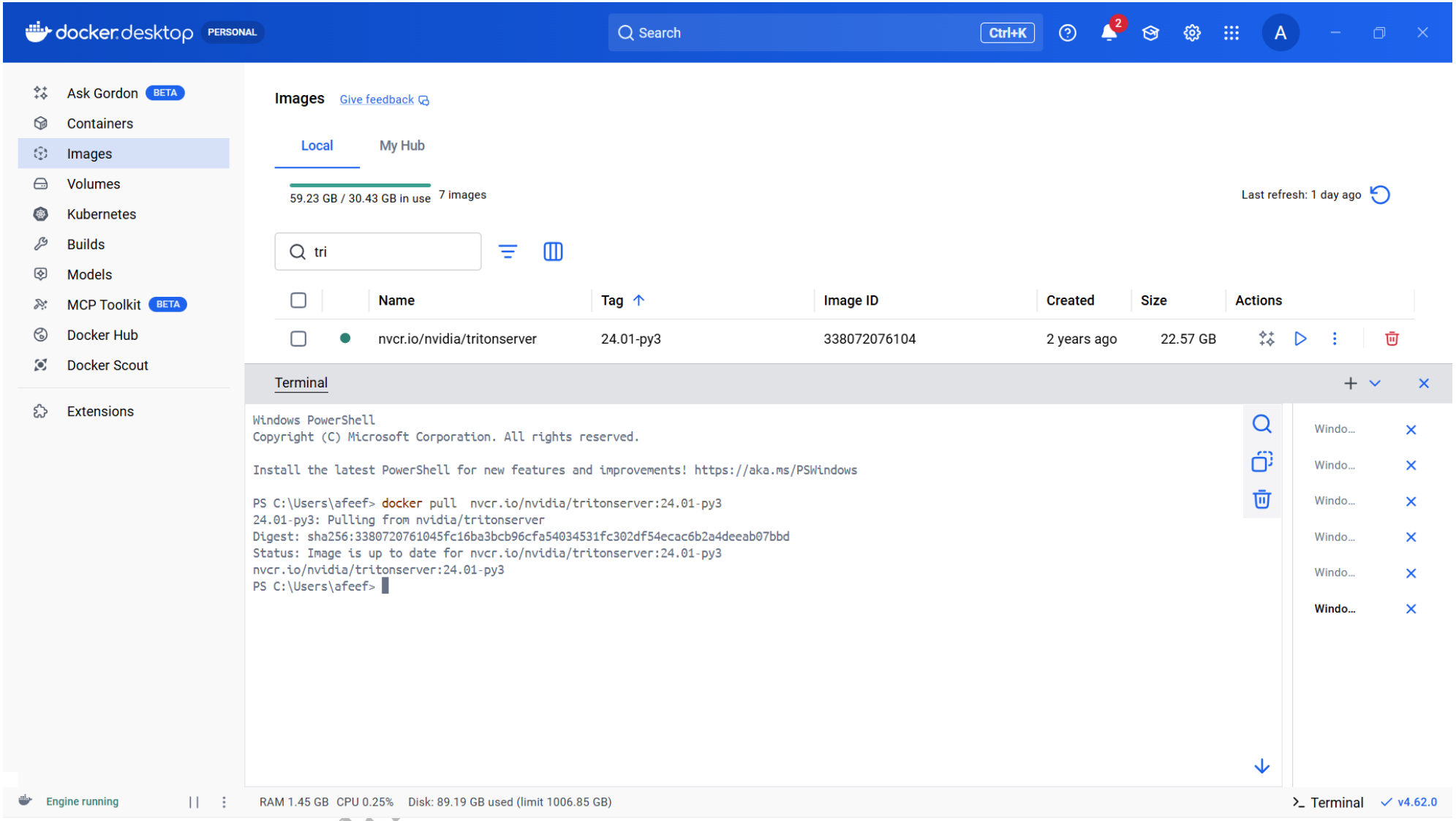

Figure 12: Use Case Container Pull

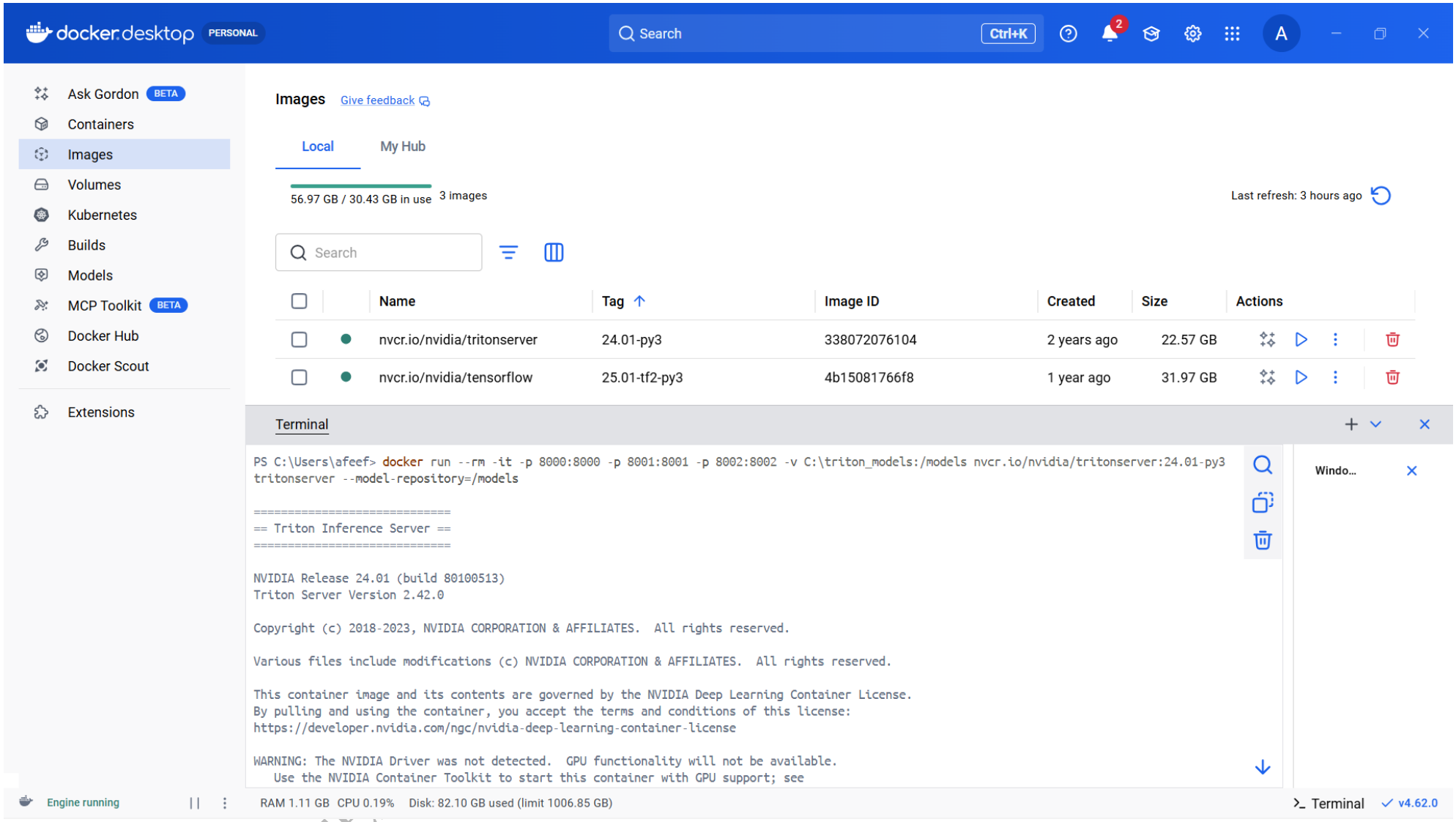

Figure 13: Use Case Container Run

Task 3 : Installition and Pulling Packages from DOCKER Hub

Date: 20-02-2026

Self Project Pushed in Docker Hub

The personal project was containerized and uploaded to Docker Hub, so that it can be shared, deployed, and run on any machine with Docker.

First, a Docker image was created from the project directory using a Dockerfile with this command:

```
build -t fitnessapp .
```

After the successful building of the image, it was tagged with the Docker hub username and version number, thus preparing the image for the upload:

```
docker tag fitnessapp afeefahmed/fitnessapp:v1
```

Next, the Docker Hub authentication was done by running:

```
docker login
```

Once logged in, the image was pushed to the Docker Hub repository using:

```
docker push afeefahmed/fitnessapp:v1
```

During the push operation, Docker uploaded the image layers to the Docker Hub repository. The digest message indicated that the upload was successfully completed.

This finalizes the assurance that the containerized project can be downloaded and run on any computer system, thus ensuring portability and deployment ease.

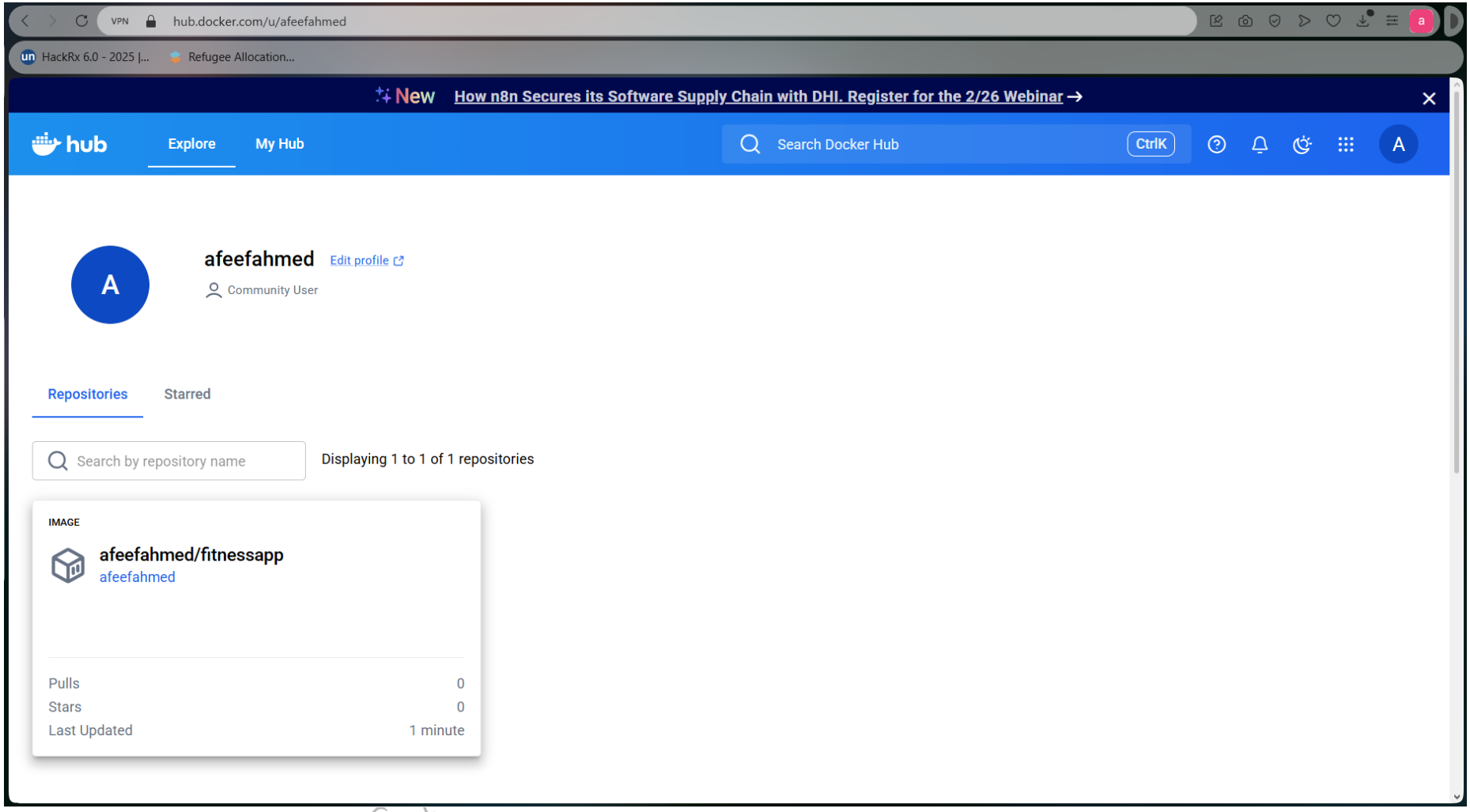

Figure 14: Self Project Pushed in Docker Hub

Pulling Self Image and Running Self Image

The project image that was previously uploaded to Docker Hub was pulled again to verify that it can be downloaded and run on any system. This is to ensure that the project image can be downloaded and run on any machine.

The image was downloaded via the below command:

```
Docker pull afeefahmed/fitnessapp:v1
```

After downloading, there was a message in Docker that the image was successfully received or was already available in the latest version.

Finally, the container was launched to run the application:

```
docker run afeefahmed/:v1
```

However, since the project is a web-based application, it was run with port mapping so that it could be accessed via a web browser:

```
docker run -p 8600:8600  afeefahmed/fitnessapp:v1
```

Once the container had started running, the application was available at: Once the container was running, the application was available under the following address: http://localhost:8600

This proved that the project image was indeed pulled from Docker Hub and ran successfully, proving container portability and Docker's ability to perform rapid deployment across systems.

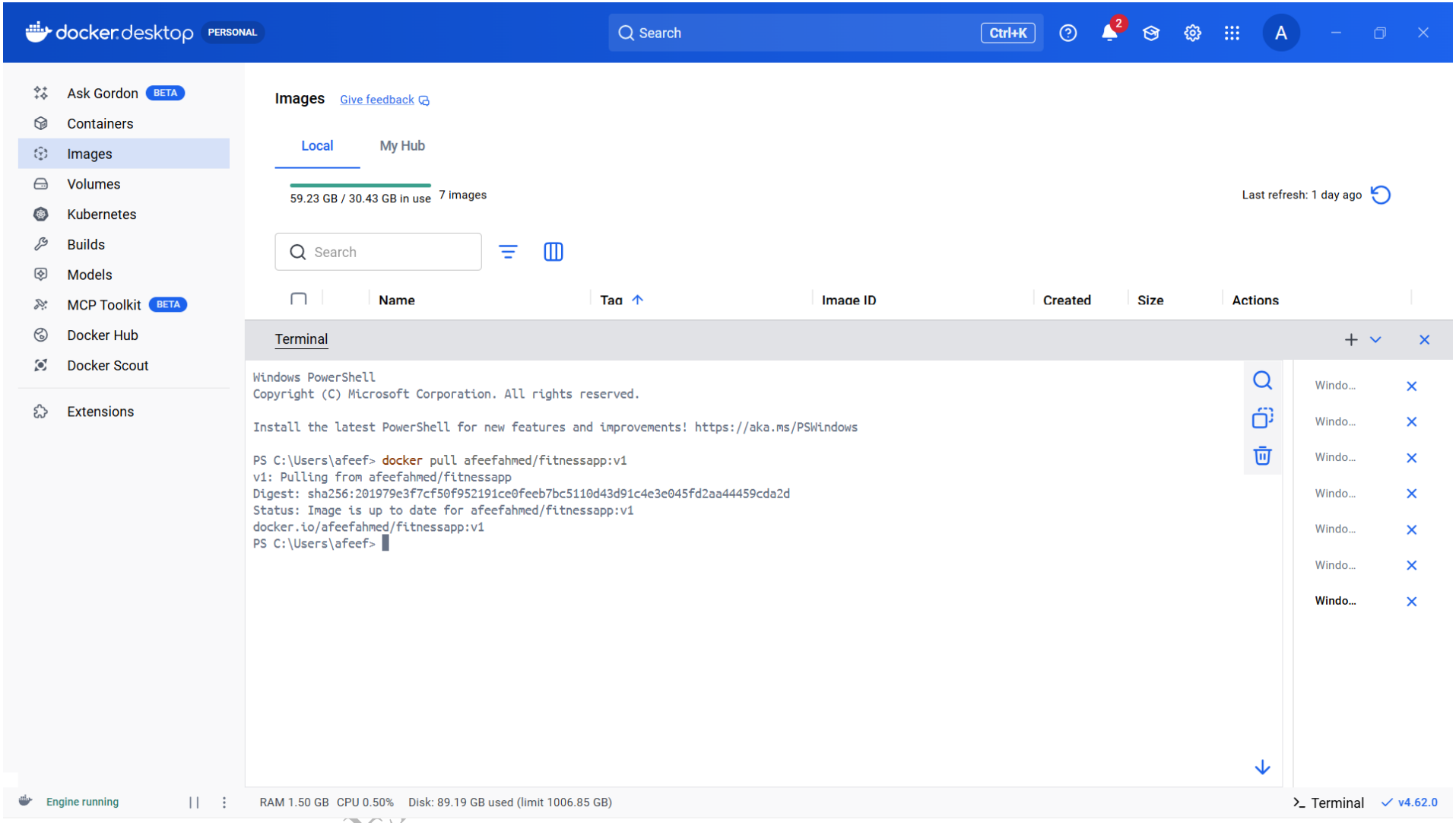

Figure 15: Pulling Self Image

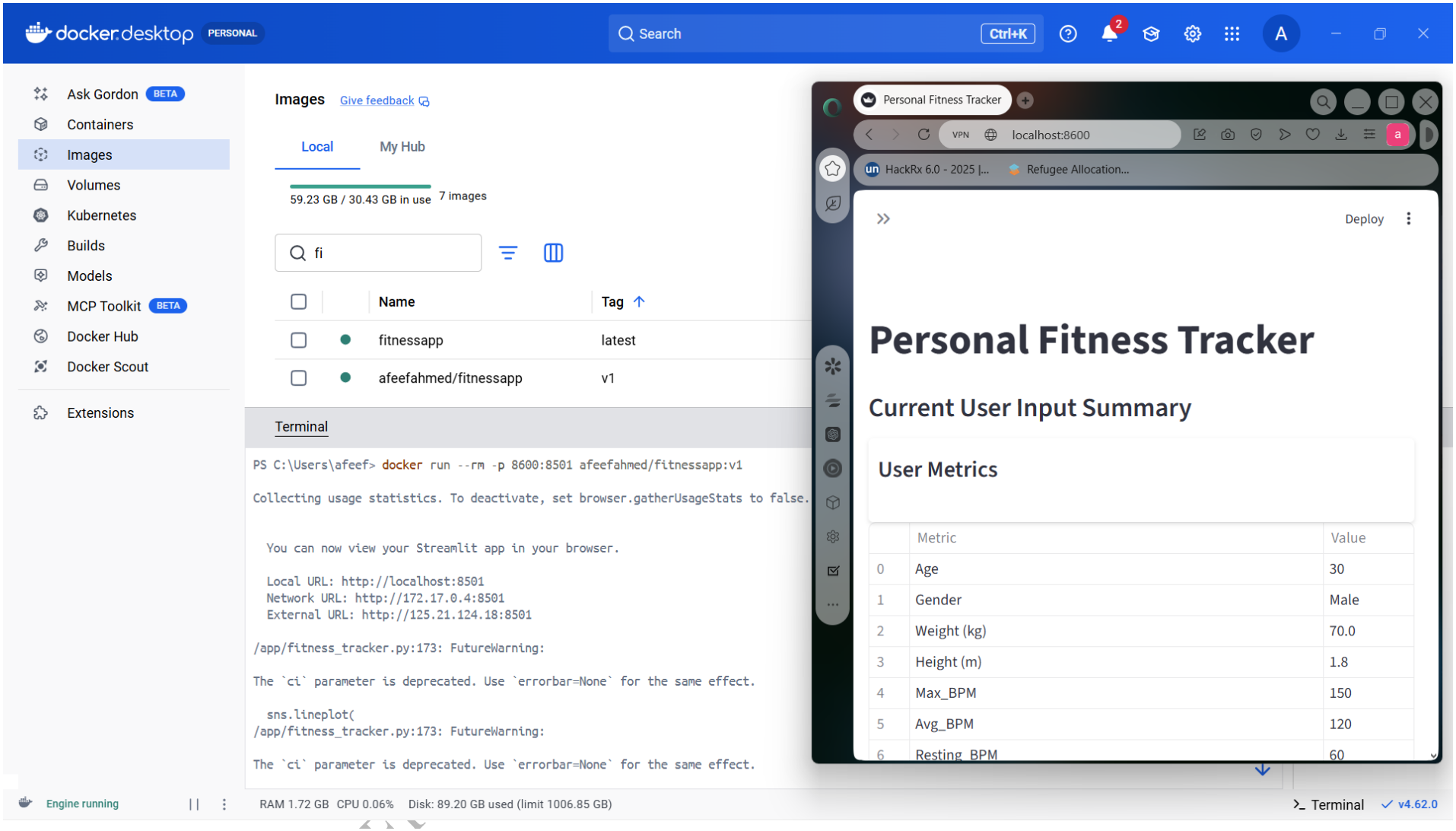

Figure 16: Running self-image

Pulling and Running Friend Image

The docker image shared by a friend was pulled from the docker hub to verify that containers published by other users can be successfully downloaded and executed. Pulling the image ensures that a similar version of the application can be created on other machines without any need for manual installation or setup.

The image was downloaded using the following command:

```
docker pull afeefahmed/fitnessapp:v1
```

After the process was complete, a confirmation message was displayed by Docker, confirming that the image had been successfully retrieved.

Next, the application was launched by executing the container:

```
docker run afeefahmed/fitnessapp:v1
```

If the application involves a web-based interface, it can be started by port mapping to enable browser access:

```
docker run -p 8600:8600 afeefahmed/fitnessapp:v1
```

However, the application worked fine after the container was put into operation. Once the container was started, the application was running successfully, which proved that containers created by other users can be pulled and executed, showing container portability and container sharing through Docker Hub.

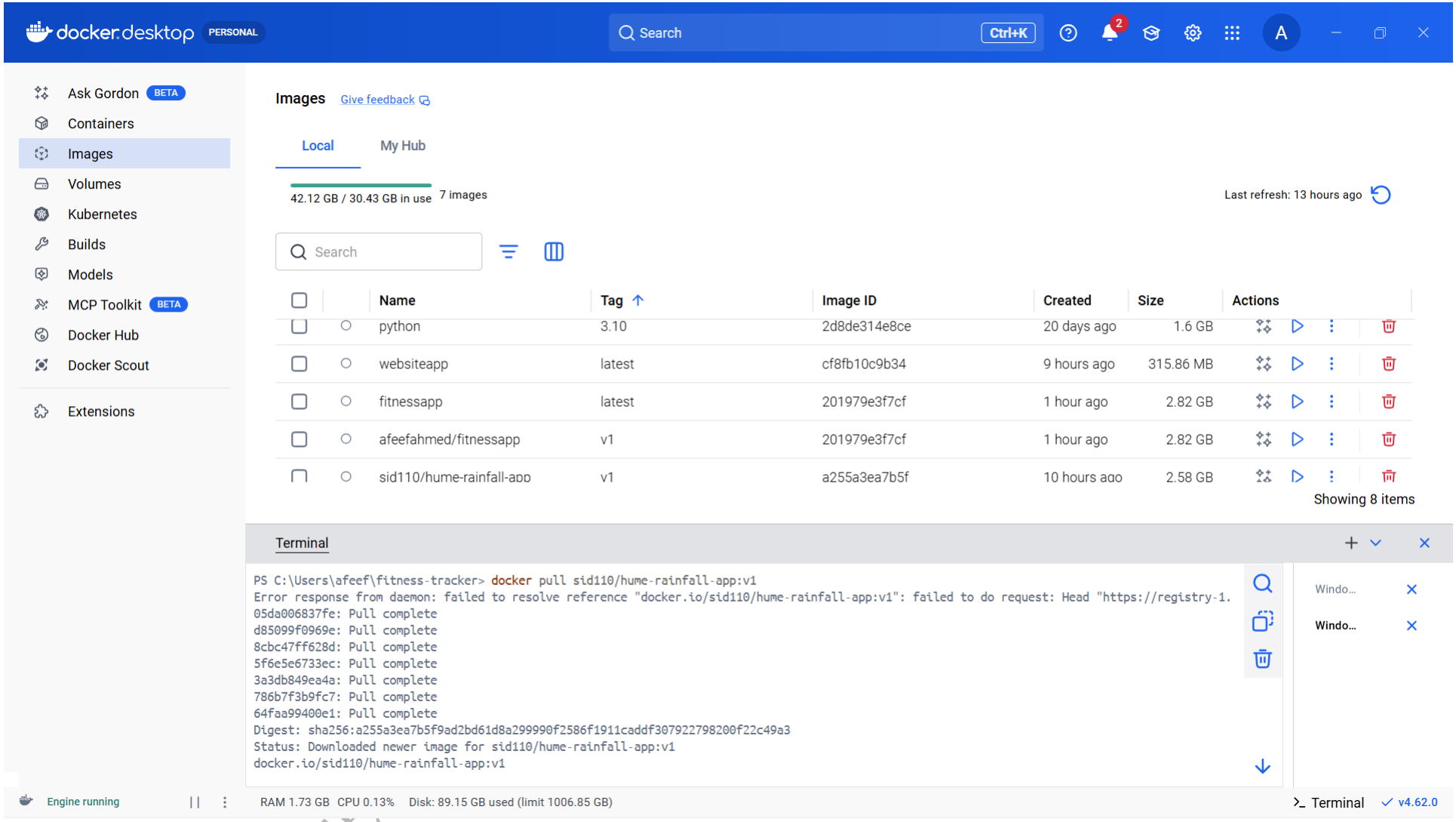

Figure 17: Pulling Friend Image

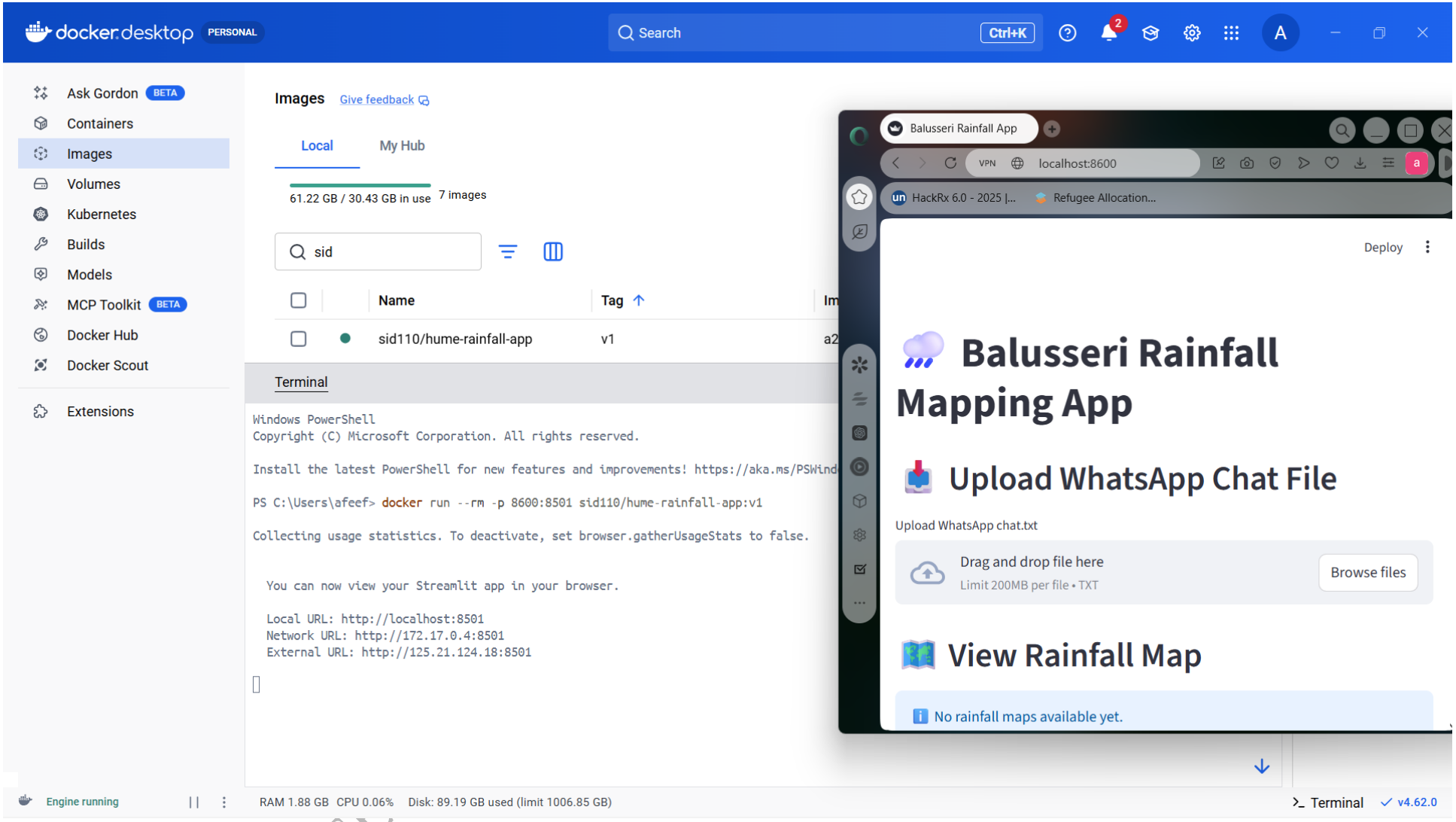

Figure 18: Running Friend Image

Docker Images After All Pulls

After downloading all the necessary containers, the list of Docker images was checked to ensure that every image was successfully pulled and stored locally. Finally, the list of docker images was checked to make sure that all images have been downloaded and stored locally, which includes tensorflow, pytorch, triton server images, self project image, and friend's image.

The below command was run:

```
docker images
```

This command prints all the locally saved Docker images alongside some major details, including:

- Repository
- Tag (version)
- Image ID
- Image creation date
- Size of the image

Output images included:

- NVIDIA TensorFlow container
- NVIDIA PyTorch container
- Triton Inference Server container
- A self project image retrieved from Docker Hub (ibid).
- A friend's shared Docker image

Finally, this verification step assured that:

- All of the required images were downloaded successfully
- The images are downloaded and saved locally (Docker environment)
- The containers are in place and can be run when necessary

Hence, this confirms successfully completing the image pulling process and correctly setting up the Docker environment.

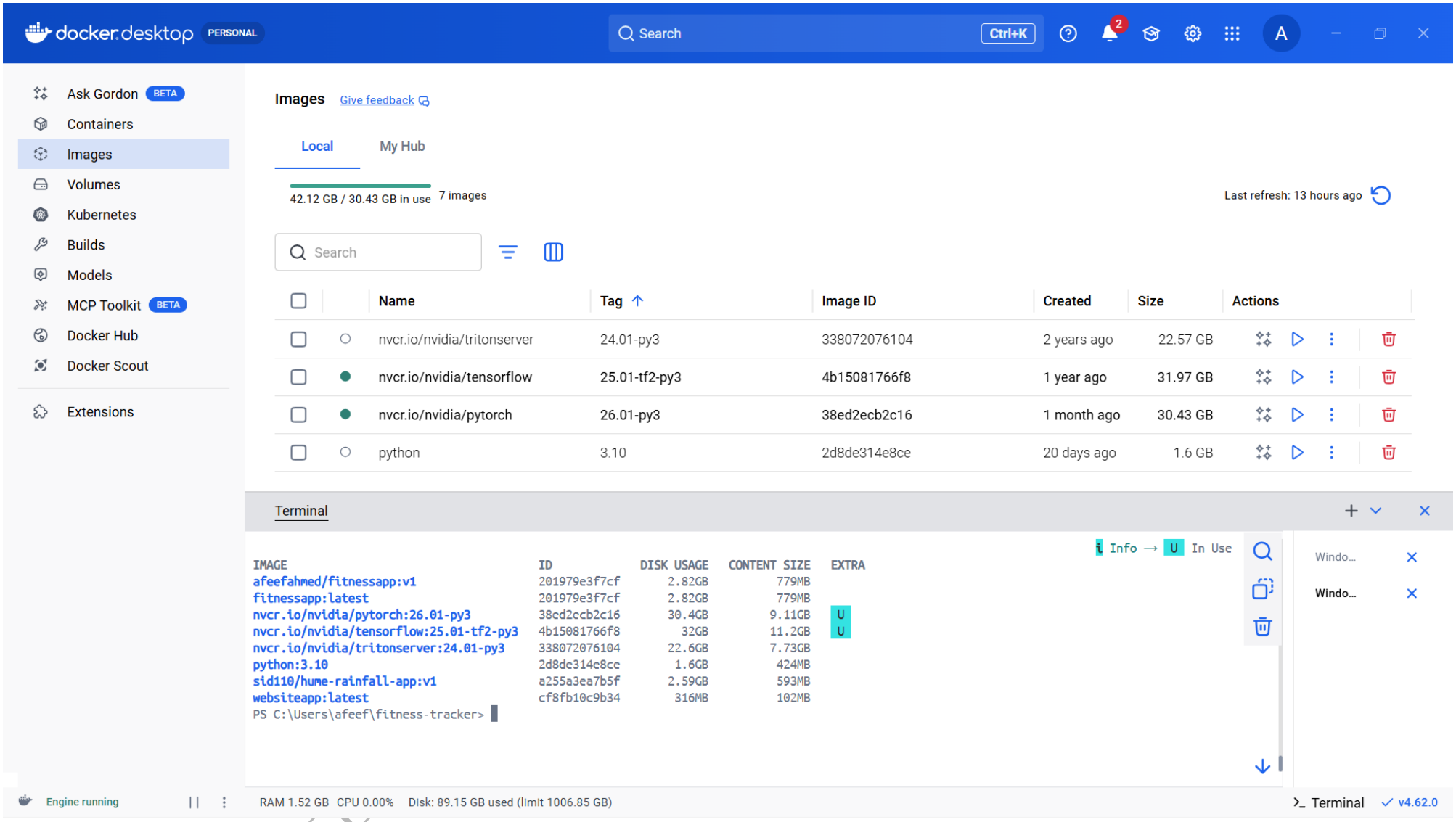

Figure 19: Docker Images After All Pulls

Task 4 : Implementing and executing Kubernetes

Date: 27-02-2026

CHECKING NODES & PODS AFTER INSTALLING MINIKUBE

After installing Minikube, a Kubernetes cluster was started to build a local single-node cluster for running containers.

The cluster was started through the command:

```
minikube start
```

This command executes the following functions:

- Creates a local Kubernetes cluster
- Sets up a virtual node using the Docker driver
- Configures kubectl communication with the cluster (ibid)

Once the startup procedure was complete without any errors, the cluster status was checked.

For checking whether the node was running correctly, the following command was fulfilled:

```
kubectl get nodes
```

The output showed the minikube node as Ready, indicating that the cluster was correctly initialized.

Next, all pods in the running status across the namespace are checked using:

```
kubectl get pods -A
```

This command outputted relevant system pods necessary for Kubernetes operations, including:

- kube-apiserver
- kube-controller-manager
- kube-scheduler
- Other core Kubernetes services

These verification steps proved that:

- Minikube was successfully started
- The Kubernetes node was in a Ready
- Verification that all system's components were in order

This made sure that the local kubernetes cluster was fully functional and ready to be used for deploying containerized applications.

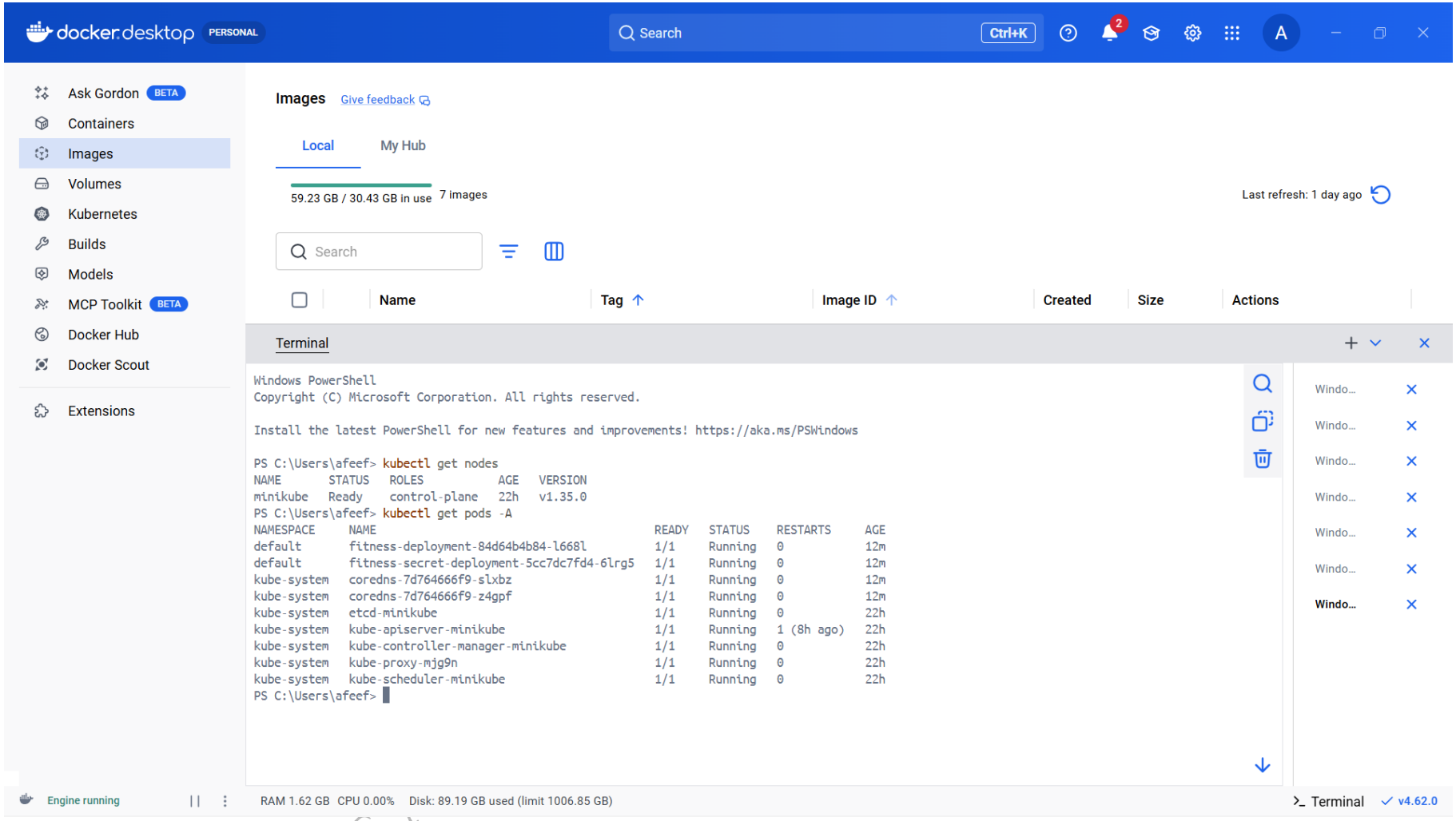

Figure 20: Checking Nodes & Pods After Installing

YAML DEFINING FOR SELF PULL

A Kubernetes deployment YAML file was created to deploy the self project image from Docker Hub into the Minikube cluster. This configuration file states the desired state of the application, and it includes details, such as the container image, number of replicas, and ports.

When this YAML file gets applied, Kubernetes, by default:

- Pulls the required container image from Docker Hub
- Sets up the pods to the number specified in the configuration
- Maintains the number of replicas as specified
- Guarantees that the application runs as per the configuration

The deployment configuration was saved in a file called:

```
apiVersion: apps/v1
kind: Deployment
metadata:
 name: fitness-deployment
spec:
 replicas: 1
 selector:
  matchLabels:
   app: fitnessapp
 template:
  metadata:
   labels:
    app: fitnessapp
  spec:
   containers:
   - name: fitness-container
    image: afeefahmed/fitnessapp:v1
    ports:
    - containerPort: 8501
```

This configuration is meant to instruct Kubernetes to create a single replica of the container from self project image from Docker Hub. Upon the deployment being applied, Kubernetes downloads and executes it its cluster (on devices contained in the cluster) in case the image has not been previously downloaded. Thus, the application is run according to this configuration inside the Minikube (Kube) environment.

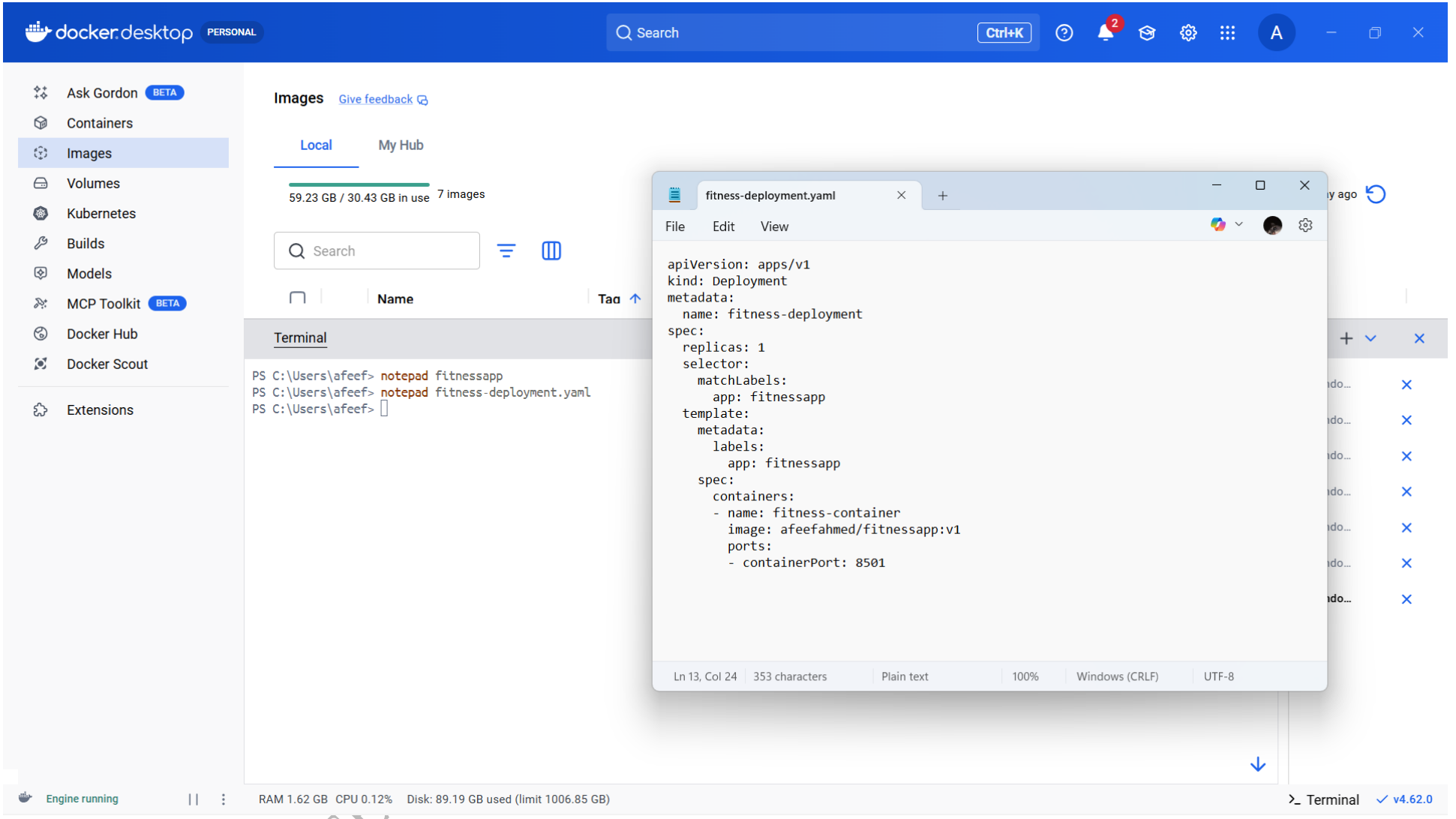

Figure 21: Yaml Defining For Self Pull

SELF PULL EXECUTION

After preparation of the deployment YAML file, it was deployed on the Minikube cluster to deploy the self project image. This step instructs the Kubernetes to create the pod by pulling the image from Docker Hub.

The deployment was done using the following command:

```
kubectl apply -f fitness-deployment.yaml
```

Below are the actions performed by Kubernetes upon the run of the above command:

- Created a deployment resource
- Pulled the container image from Docker Hub (if not already available)
- Started the container in the pod

To confirm that the pod was successfully created and running, the following command was applied:

```
kubectl get pods
```

The output showed one pod related to the deployment in a Running state.

This verified that the self project image was pulled successfully and then run using the Minikube Kubernetes cluster.

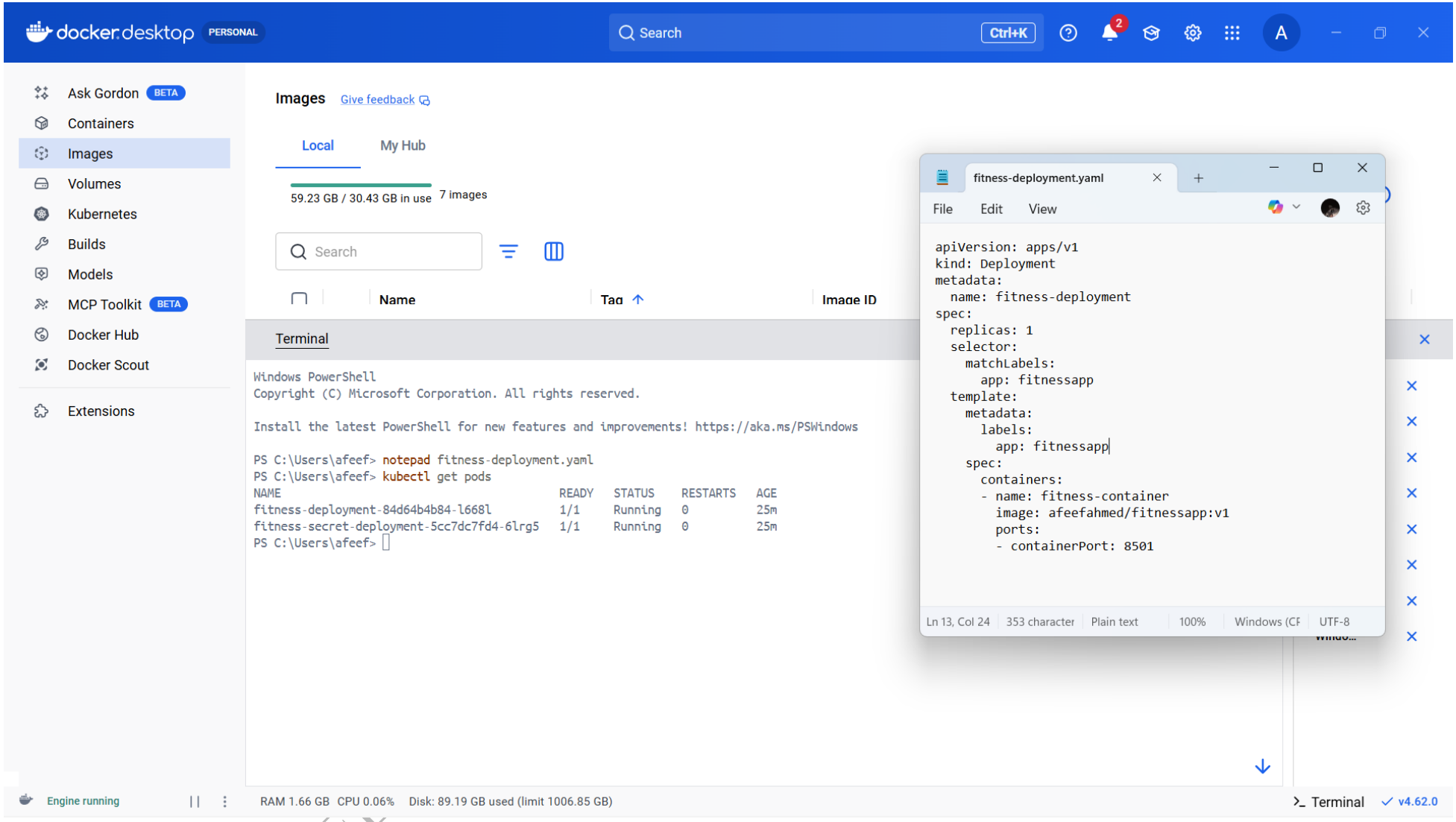

Figure 22: Self Pull Execution

UPSCALE

After deployment of the application in Minikube was successful, the deployment was scaled up to increase the number of running pods. Scaling enables Kubernetes to run multiple instances of the same application, which is useful for:

- Cater for more users
- Increase reliability through redundancy
- Enhance the availability of the application

The deployment was scaled by the following command:

```
kubectl scale deployment fitness-deployment --replicas=3
```

This command instructs Kubernetes to run three replicas of the container instead of

one. To scale out the container successfully, the following command was run to

ascertain the same:

```
kubectl get pods
```

The output showed three pods in the Running state, all linked to the deployment.

This verified that the application was scaled successfully and multiple instances were running within Minikube cluster.

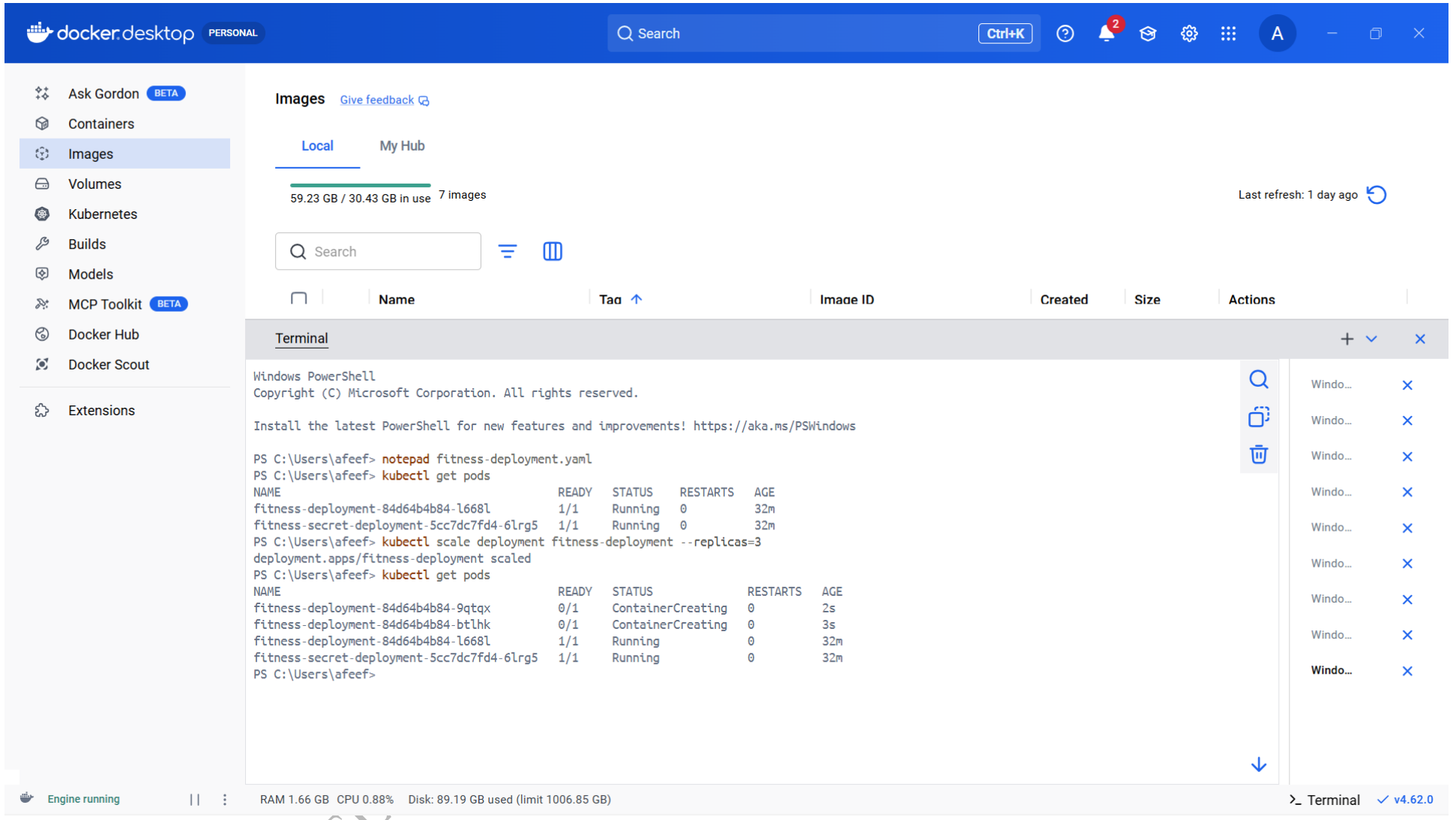

Figure 23: UPSCALE

DOWNSCALE

After increasing the number of replicas, the deployment was scaled down to demonstrate a Kubernetes downscaling. Downscaling also achieves a reduction in the number of active pods, thus releasing unnecessary system resources.

The deployment was downscaled by running the below command:

```
kubectl scale deployment fitness-deployment --replicas=1
```

This command tells Kubernetes to scale down the number of replicas to one.

To ensure that the downscaling operation was carried out successfully, the following command was run:

```
kubectl get pods
```

There was only one Running pod in the output, indicating that the deployment had been scaled down to one replica as intended.

This demonstrated Kubernetes' ability to dynamically create the number of instances of an application required at any given time.

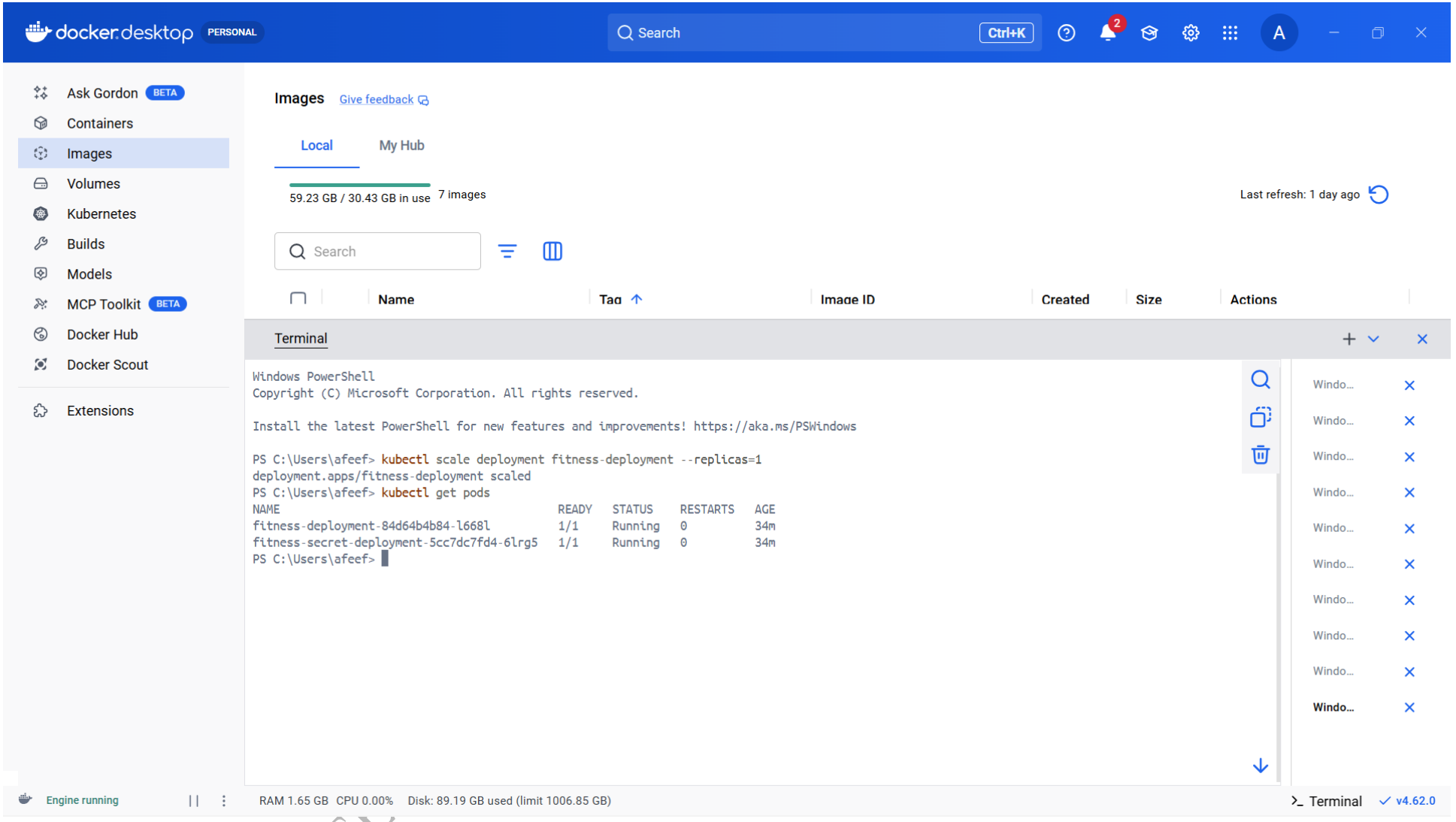

Figure 24: Downscale

SECRET YAML DEFINING

A Kubernetes Secret YAML file was created to hold the configuration data securely within the Minikube cluster. Secrets are useful to store passwords, API keys, or environmental variables without hardcoding them into the application.

The secret configuration was saved into a file called:

```
apiVersion: v1
kind: Secret
metadata:
  name: fitness-secret
type: Opaque
data:
  username: YWRtaW4=
  password: MTIzNA==
```

Here is the configuration:

- A Secret named app-secret is created
- The type Opaque defines a generic secret
- The key-value pair has APP_MODE set to production
- he stringData field accepts non-encoded data, which Kubernetes re-encodes using Base64 internally

This secret can later be referenced in a deployment file, usually as an environment variable in a container. This approach secures the sensitive configuration with a clear distinction from application code.

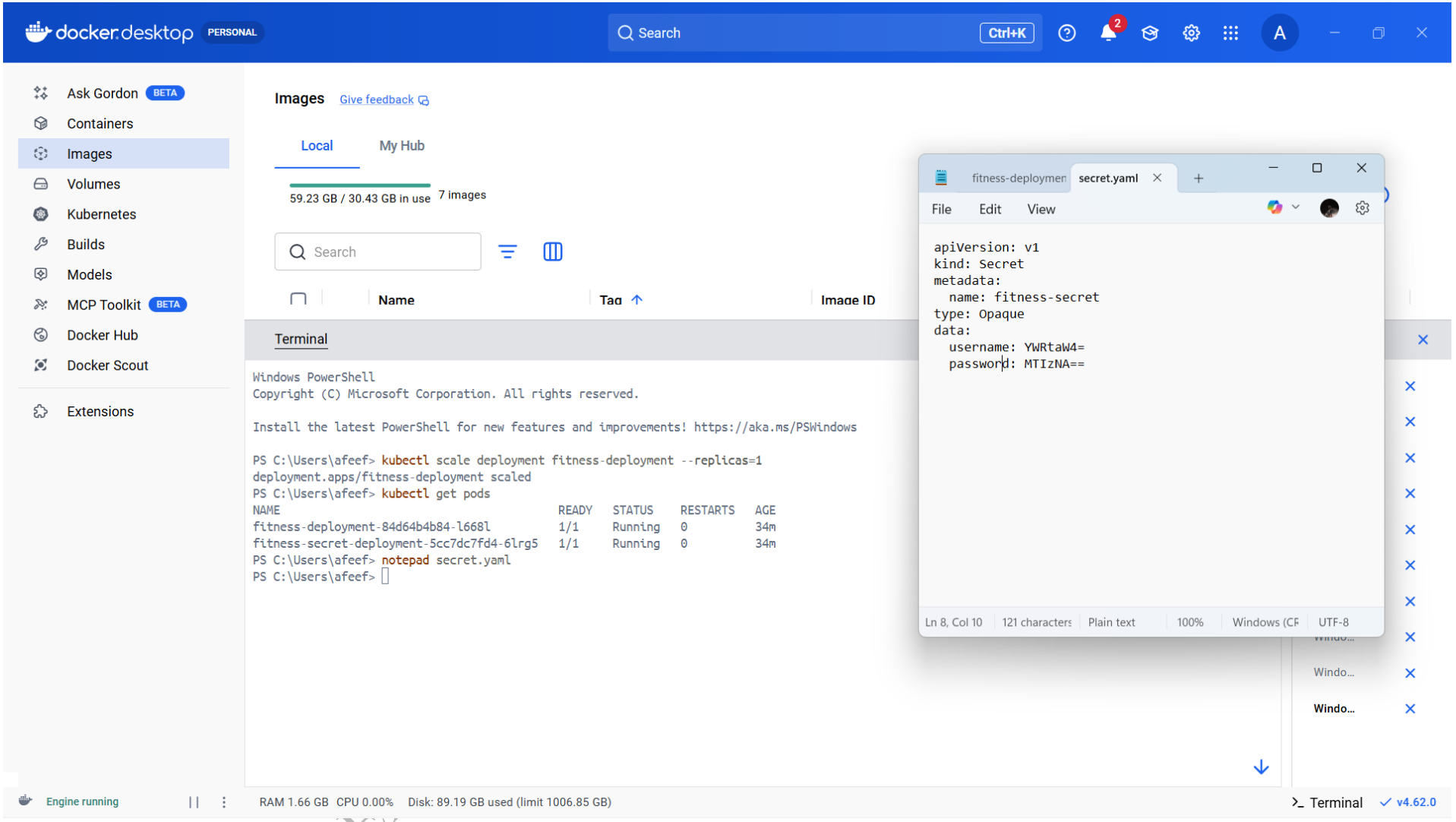

Figure 25: Secret Yaml Defining

NEW DEPLOYMENT DEFINING KIND SECRET

To show how a Secret can be passed into an app via environment variables, a new Kubernetes deployment configuration was done. This deployment injects Secret value into a running container instead of hardcoding it into an application.

The configuration was saved in a file called:

```
apiVersion: apps/v1
kind: Deployment
metadata:
 name: fitness-secret-deployment
spec:
 replicas: 1
 selector:
  matchLabels:
   app: fitnesssecret
 template:
  metadata:
   labels:
    app: fitnesssecret
  spec:
   containers:
   - name: fitness-container
    image: afeefahmed/fitnessapp:v1
    ports:
    - containerPort: 8501
    env:
    - name: USERNAME
     valueFrom:
      secretKeyRef:
       name: fitness-secret
       key: username
    - name: PASSWORD
     valueFrom:
      secretKeyRef:
       name: fitness-secret
       key: password
```


Explanation of the Configuration

- A deployment named fitness-secret-deployment is created
- t runs a single replica container based on the self project image from Docker Hub.
- The selector and labels are in place to ensure the proper identification and handling of the associated pods by Kubernetes.
- The vital part is the env section, where:
    - There is a set definition for the environment variable APP_MODE.
    - It is not specified directly in the file..
    - However, the value is not put directly in the file but retrieved securely from the Kubernetes Secret called app-secret.
    - The secretKeyRef defines the secret name and the particular key necessary for access.

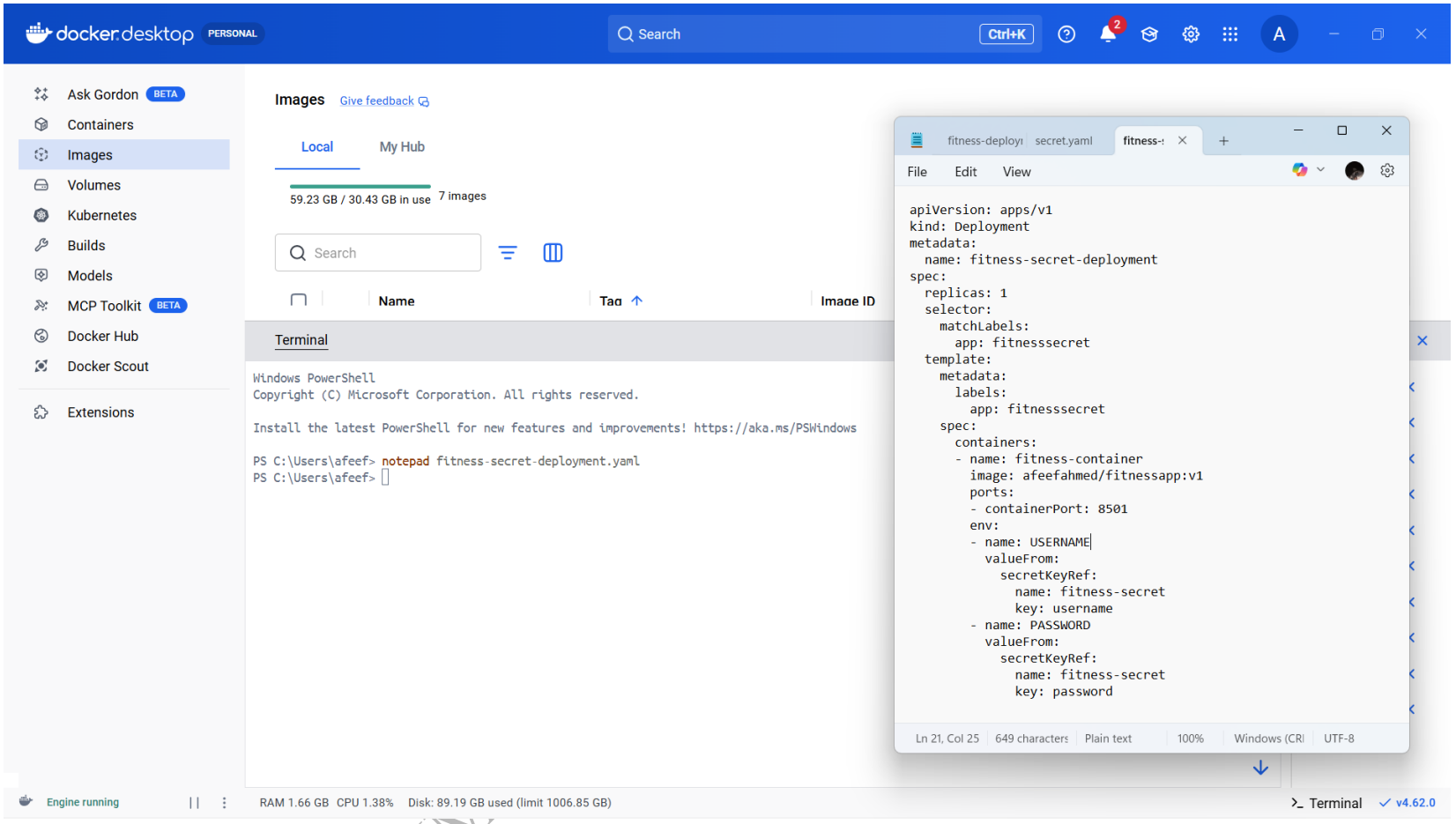

Figure 26: New Deployment Defining Kind Secret

GETTING SECRET FOR CHECK

Once the Kubernetes Secret had been generated, its presence and contents were validated by using various commands in order to prove that the Secret had been correctly stored within the Kubernetes cluster. Several commands can be used to inspect Kubernetes Secrets.

In order to list all the secrets stored within the cluster, the following command was run:

```
kubectl get secrets
```

As can be seen in the output above, there is indeed a secret called app-secret, which has been created.

In order to get additional information regarding the metadata associated with this secret, the following command was run:

```
kubectl describe secret app-secret
```

Additional metadata obtained from this secret include:

- Secret name
- The type of secret
- The number of keys stored in it

Finally, in order to validate that the entire content of this secret had been correctly stored, the following command was run:

```
kubectl get secret app-secret -o yaml
```

The stored values are shown in Base64 encoded format. In order to actually get the value of the APP_MODE field, the following command was used:

```
kubectl get secret app-secret -o jsonpath="{.data.APP_MODE}" | base64 -- decode
```

Conclusion

The verification steps shown in this output demonstrate that:

- The Kubernetes secret has been successfully created
- It has been securely stored within the Kubernetes cluster
- he values stored within it can be accessed when necessary

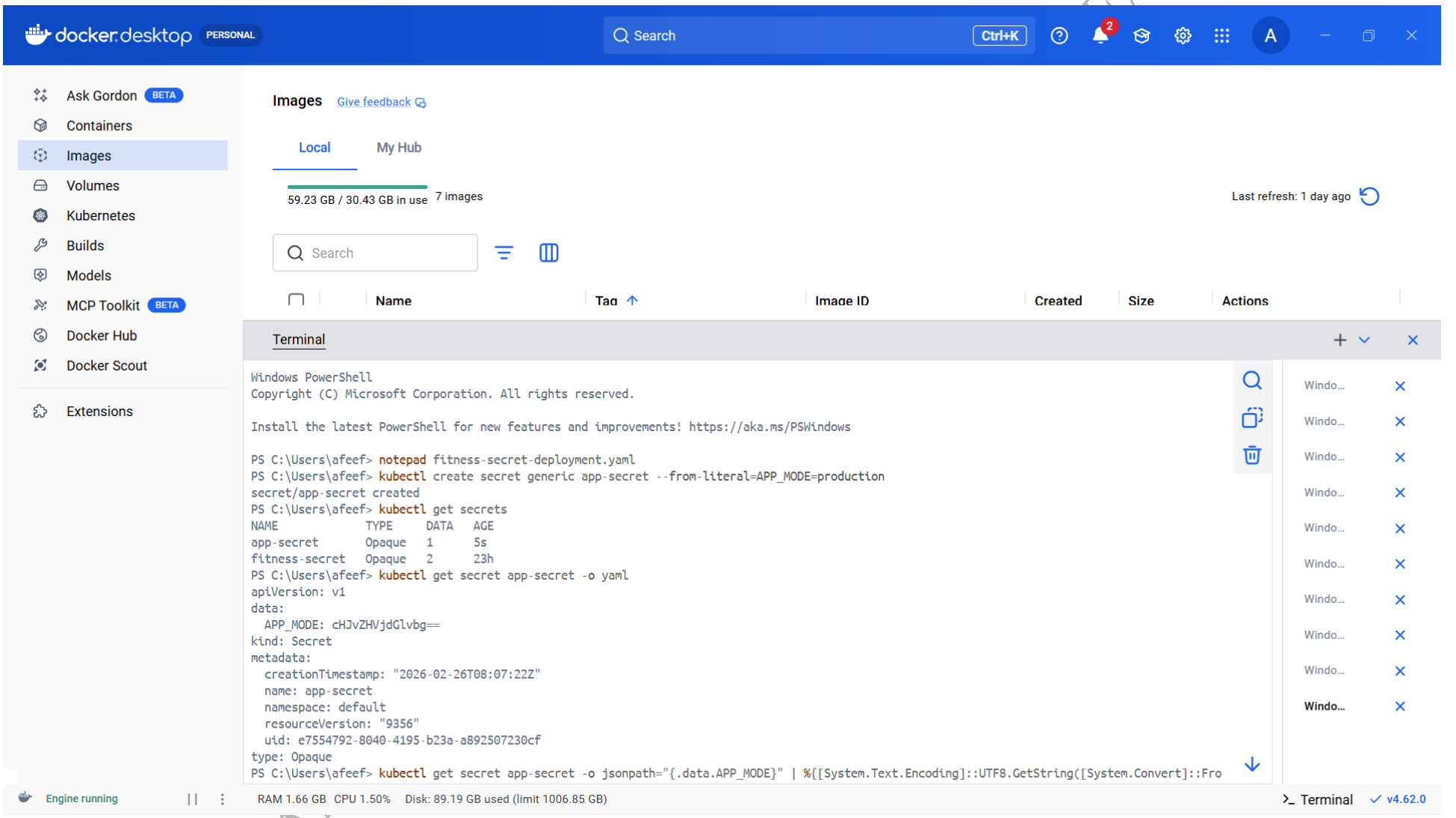

Figure 27: Getting Secret For Check

SECRET DEPLOYMENT

Once the Kubernetes secret had been created, its creation and storage were validated in order to demonstrate proper creation. There are many commands available within Kubernetes that could verify the secret in question.

Firstly, the following command was used in order to list all the secrets currently available within the Kubernetes cluster:

```
kubectl get secrets
```

It shows that there is a secret called app-secret available within the cluster.

The following command was used in order to display additional metadata:

```
kubectl describe secret app-secret
```

Additional metadata revealed by this secret include:

- Secret name
- Secret type (Opaque)
- Keys stored within the secret

In order to verify the data stored within the secret, the following command was used:

```
kubectl get secret app-secret -o yaml
```

Note that the stored values are in Base64 format, because Kubernetes encrypts the secret data automatically.

kubectl get secret app-secret -o jsonpath="{.data.APP_MODE}" | base64 -- decode:

```
kubectl get secret app-secret -o jsonpath="{.data.APP_MODE}" | base64 -- decode
```

Conclusion
The verification steps demonstrated above confirm that:

- The Kubernetes secret has been successfully created
- It has been securely stored within the Kubernetes cluster
- The values stored within it can be accessed whenever required

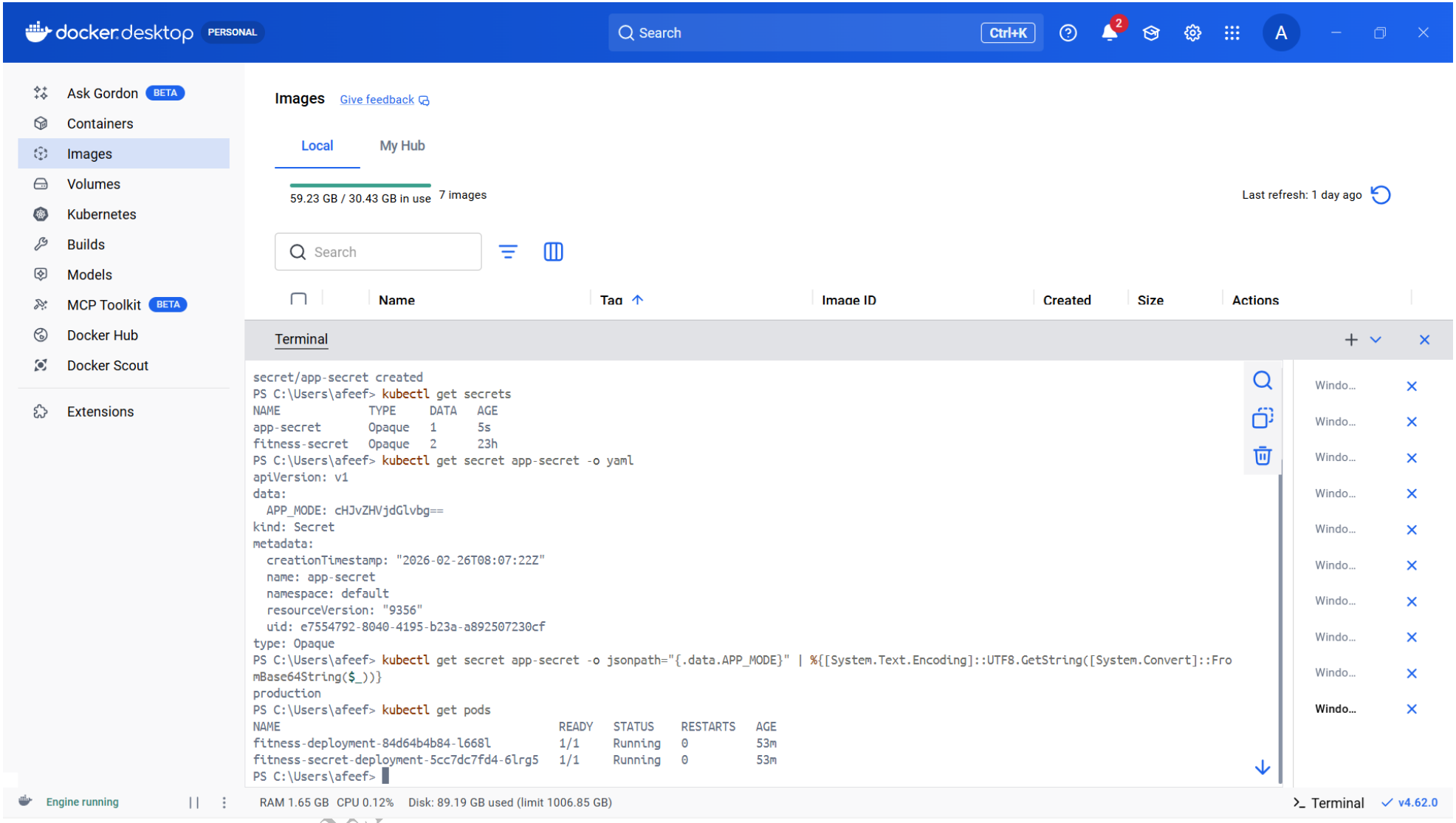

Figure 28: Secret Deployment

UPSCALE

In order to further demonstrate the capabilities of the application and the platform, the deployment was upscaled to increase the number of active pods. Upscaling makes the application more reliable and proves that the Kubernetes platform can handle increasing workloads.

The deployment is upscaled using the following command:

```
kubectl scale deployment fitness-secret-deployment --replicas=3
```

This instruction tells Kubernetes to make three copies of the pods related to this deployment.

To verify whether the deployment is upscaled successfully, the following command is executed:

```
kubectl get pods
```

As a result, three pods with the status of Running are observed with the name of fitness-secret-deployment.

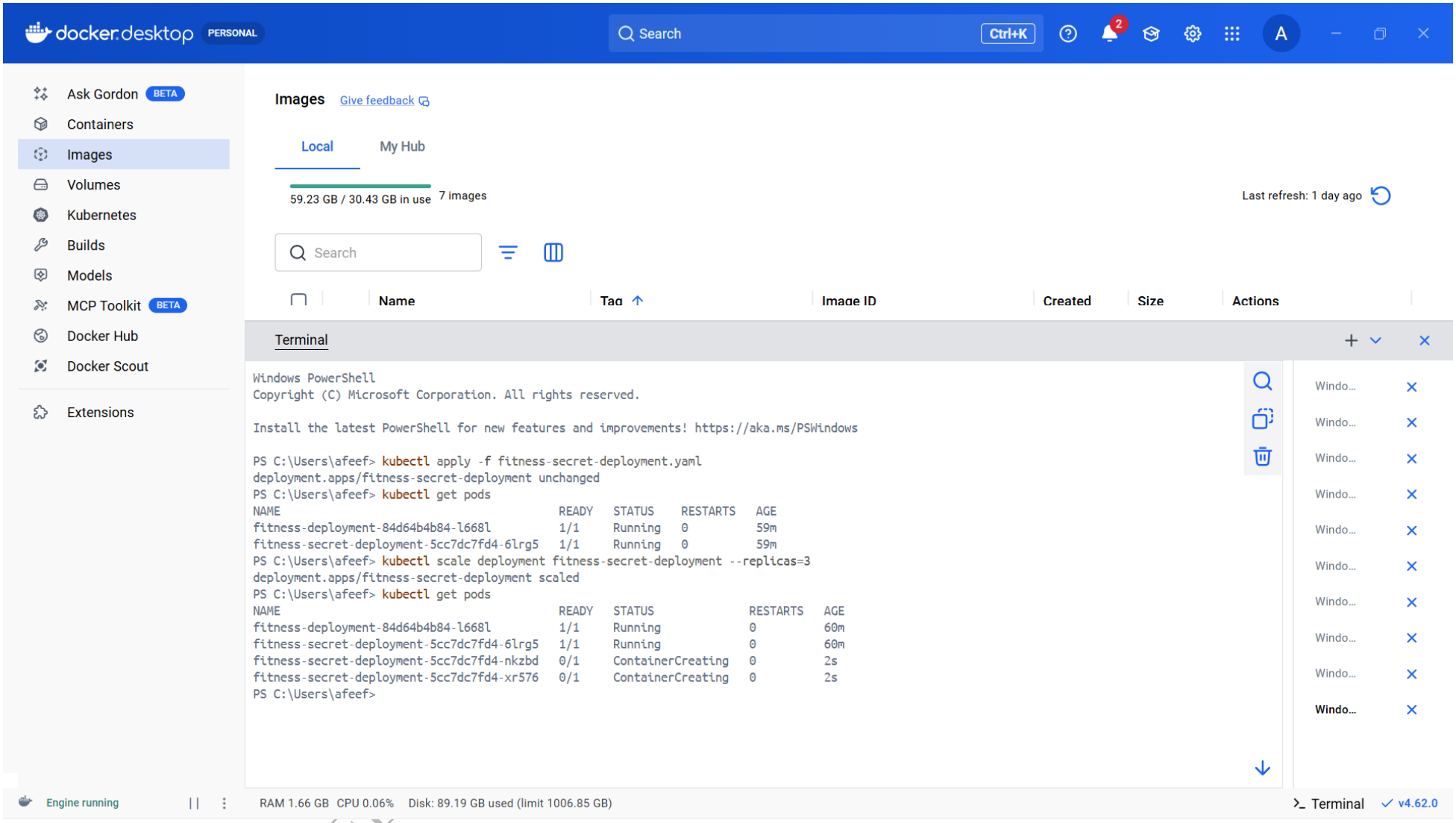

Figure 29: UPSCALE

DOWNSCALE

Having demonstrated the upscaled deployment, the deployment was downscaled, which means that fewer pods will be running.

Downscaling helps optimize the usage of resources and prove that Kubernetes can effectively adjust its capacity for an application.

The deployment is downscaled using the following command:

```
kubectl scale deployment fitness-secret-deployment --replicas=1
```

kubectl scale deployment fitness-secret-deployment --replicas=1

This command instructs Kubernetes to scale down to one copy of the deployment.

To verify whether the operation was successful, the following command is executed:

```
kubectl get pods
```

As a result, one pod with the status of Running is observed.

This demonstrates Kubernetes' ability to efficiently manage scaling operations while maintaining application stability.

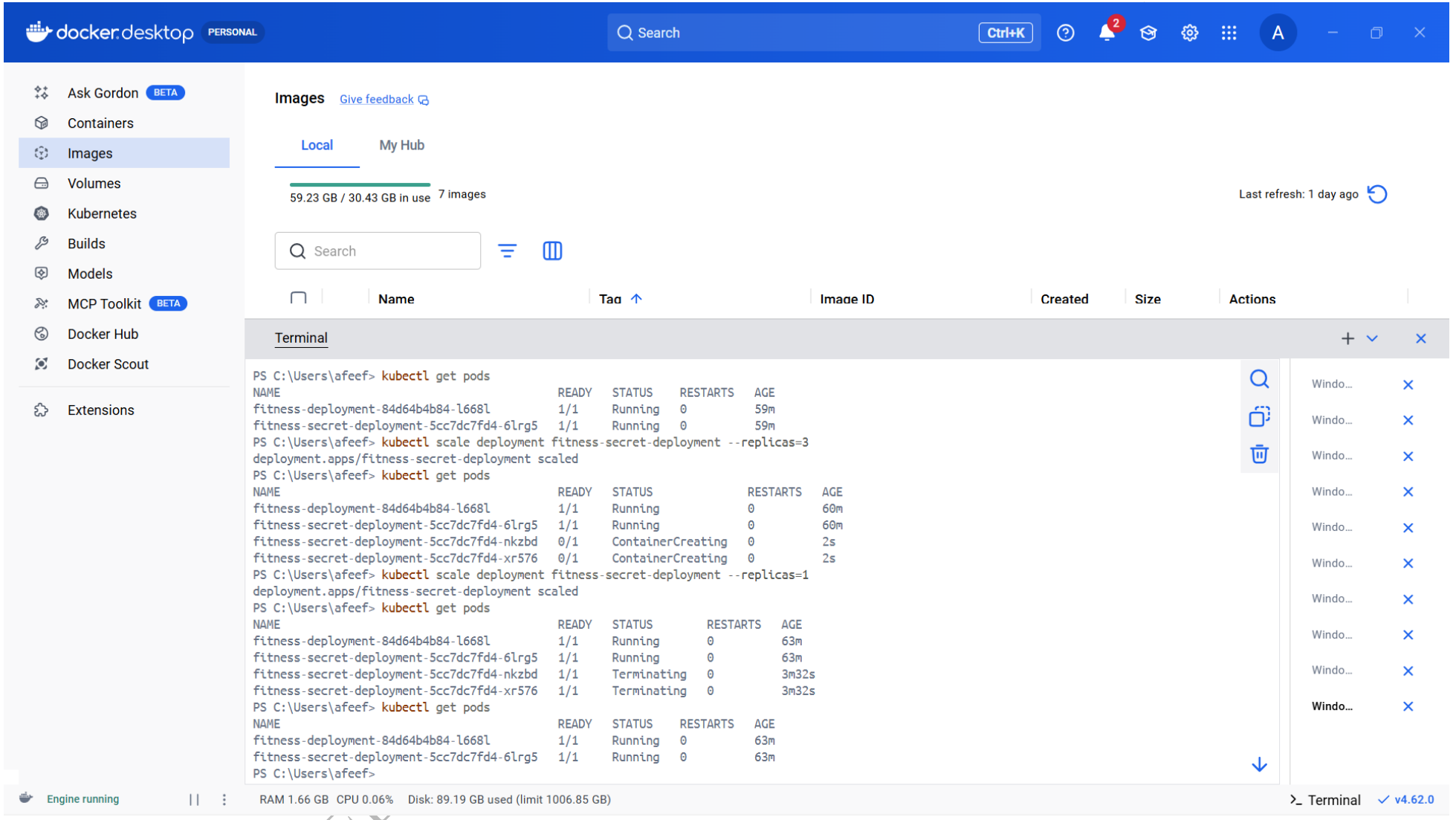

Figure 30: Downscaling

ALL FINAL DEPLOYMENT

Upon successfully completing the process of deployment, performing scaling operations, and adding secrets to the application, it is important to check all of the deployments within the cluster. This will help ensure that the applications are successfully deployed in the cluster.

The following command will show all of the deployments within the cluster:

```
kubectl get deployments
```

The output will show the following information regarding the deployment:

- Name of the deployment
- Desired number of pods
- Available pods
- status

There are deployments such as:

- **Fitness deployment**
- **Fitness secret deployment**
- **Hume rainfall deployment**

All of these deployments have the necessary number of pods running in the READY state.
To receive more information about any deployment, the following command may be executed:

```
kubectl describe deployment fitness-secret-deployment
```

The output will show:

- Information about the pod template
- List of replica sets
- Scaling events and updates.

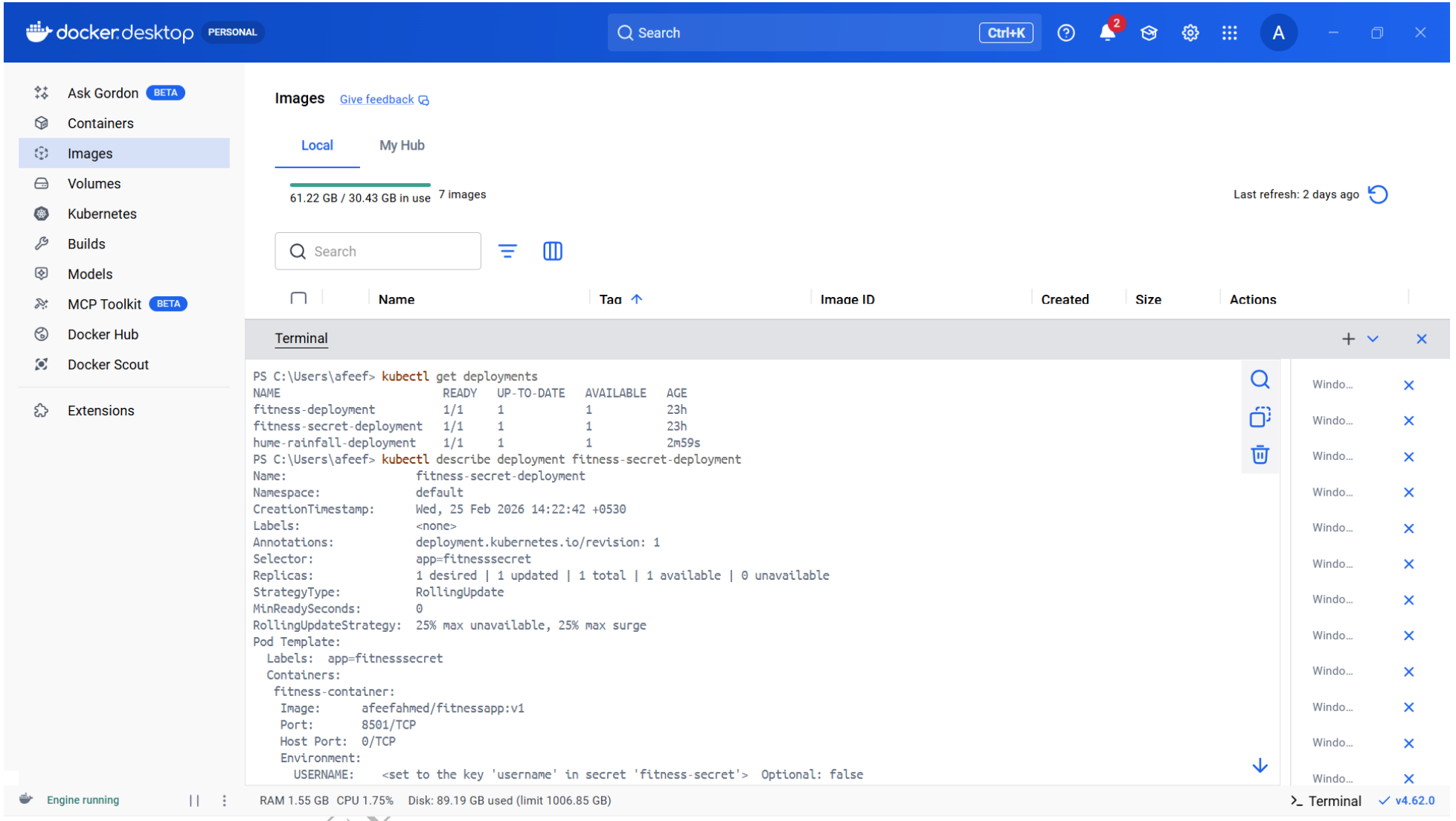

Figure 31: All Final Deployment

ALL PODS AT THE LAST

Once all deployment, scaling, and secret operations have been completed, the status of all pods in the Minikube cluster was then assessed to determine whether the applications have run smoothly and have been managed effectively.

Listing all running pods is made possible using the following command:

```
kubectl get pods
```

The following are some features displayed by the output of this command:

- Name of the Pod
- Readiness of the Pod
- Status of the Pod
- Number of restarts of the Pod
- Age of the Pod

Examples of pods include those associated with:

- **fitness-deployment**
- **fitness-secret-deployment**
- **hume-rainfall-deployment**

All the pods in this assessment had the status “Running,” with the correct number of running containers (e.g., 1/1).

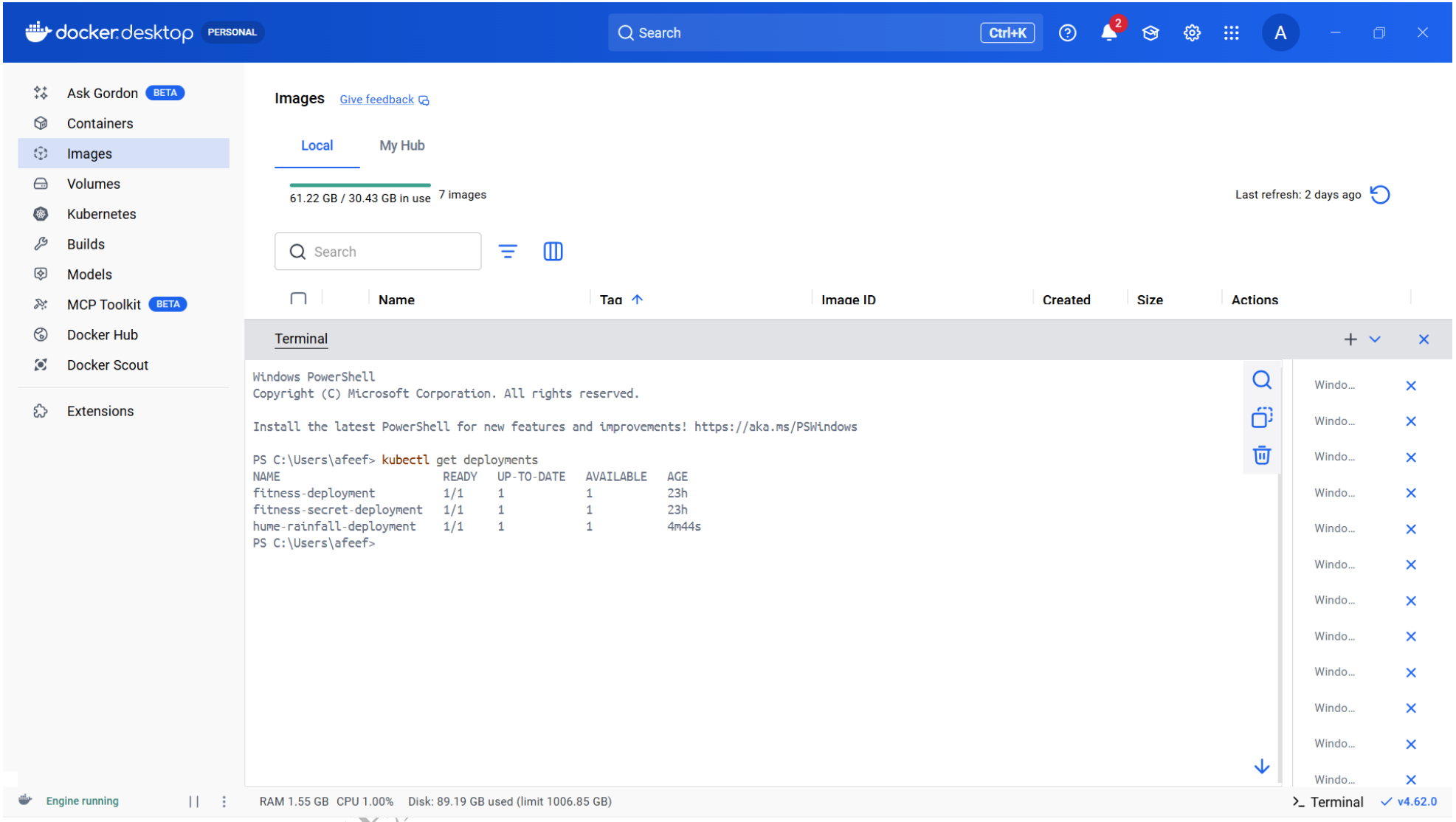

Figure 32: All Pods